From n-grams to Subquadratic Attention

A deep-dive primer on language models, scaling laws, and the post-transformer landscape

Tina Rezvanian

May 11, 2026

From n-grams to Subquadratic Attention

A deep-dive primer on language models, scaling laws, and the post-transformer landscape

How to read this book

This book is a primer for an engineer who has touched a neural network at least once and is comfortable with the words *token*, *transformer*, and *loss function*, but who would like a complete picture of *why* the field looks the way it does in 2026 — why every frontier lab is publishing a paper about *long context*, why *sparse* and *linear* attention keep coming back, and what a startup like Subquadratic is actually claiming when it says it has a *subquadratic sparse attention* mechanism.[1]

[1] The companion video lives in the same repository under `vid/scenes/`; this book is the long form of the same argument.

The ambition is to start from first principles and end somewhere uncomfortable. By the last chapter, you should be able to (a) write down on a napkin why n^2 is unavoidable for dense attention, (b) draw the 2×2 frame that positions every alternative architecture against it, (c) read a benchmark table for a long-context language model and not be fooled by it, and (d) hold an informed opinion about what could replace attention in five years.

The road map. Six parts.

- **Part I — Background.** What an LM is in one equation. Tokens (BPE, WordPiece, SentencePiece). And the pre-transformer history that almost everyone has forgotten: n-grams, RNNs, LSTMs, GRUs, sequence-to-sequence, and Bahdanau attention as a small bolt-on that ate the world.
- **Part II — The transformer.** The Vaswani recipe in detail. Self-attention from scratch, with the matrix shapes everyone gets wrong. Why the cost is exactly $\mathcal{O}(n^2 d)$ per head per layer.
- **Part III — Scaling laws.** Kaplan's straight line on a log-log plot. Chinchilla's correction. What scaling laws *don't* say about inference and context length, which is the entire reason this book exists.
- **Part IV — Living with $\mathcal{O}(n^2)$.** FlashAttention, KV caches, multi-query and grouped-query attention, paged caches, sliding windows, RoPE/YaRN. Brilliant engineering that does not change the asymptote.
- **Part V — Subquadratic alternatives.** Fixed-pattern sparse, state-space models

(Mamba), long convolutions (Hyena), kernel-feature linear attention, hybrids (SAMBA, Jamba, Griffin). What each one wins, and what each one gives up.

- **Part VI — The 2×2 frame and SubQ.** The single picture that makes the whole landscape make sense. Subquadratic Sparse Attention (SSA) as one bet on the bottom-right quadrant. How to read SubQ's published benchmarks honestly, and what comes after attention.

A NOTE ON MARGIN NOTES. This book is set in the Tufte style, so short clarifications, tangents, and citations live in the margin rather than in footnotes at the bottom of the page. They are meant to be skimmed, not mandatory. The main text reads on its own.[2]

[2] Like this. If a sentence in the body could use a sanity-check, a derivation, or a pointer to a paper, look right.

tl;dr (read this if you read nothing else). Language models are next-token predictors; training fits their parameters so that real text looks likely under that factorisation. Transformers implement the conditional $p(x_t \mid x_{<t})$ with dense self-attention, which scores every query against every key — $\mathcal{O}(n^2)$ work per layer in the sequence length n. FlashAttention and the KV-cache tricks in Part IV shrink constants and memory traffic dramatically, but they do *not* remove the asymptote: at million-token contexts the bill breaks budgets anyway. Everything in Part V is an attempt to change the curve itself — sliding windows, state-space models, linear attention, hybrids — each trading away something (usually routing by position or lossy state). The organising picture is the routing × scaling quadrant in Chapter 16: Subquadratic Sparse Attention is a bet on the bottom-right cell — linear cost *and* content-dependent routing — with public benchmarks that deserve scrutiny like any vendor-reported numbers. This book is about why long context was expensive, how engineers cope today, and where the architecture might go next.

Contents

Part I

Background — what a language model actually does

Chapter 1

What a language model actually does

A LANGUAGE MODEL is a function that takes a sequence of symbols and returns a probability distribution over what symbol could come next. That is the entire idea. Everything in this book — embeddings, attention, scaling laws, KV caches, subquadratic alternatives — is engineering on top of that one definition.

1.1 The one equation you need

Let $x_1, x_2, \ldots, x_n$ be a sequence of *tokens* (Chapter 2 defines that word precisely; for now, think "approximately a word"). A language model is a parameterised distribution

$$p_\theta(x_1, x_2, \ldots, x_n) \;=\; \prod_{t=1}^{n} p_\theta(x_t \mid x_1, \ldots, x_{t-1})\,. \tag{1.1}$$

The right-hand side is the chain rule of probability.[1] The model's job is to learn the conditional $p_\theta(x_t \mid x_{<t})$ — the probability of the *next* token given everything that came before.

Training is just maximum-likelihood estimation: pick parameters θ that make a large corpus of real text look as likely as possible.[2]

Key idea. An LM is a next-token probability machine. Training is fitting that machine to a corpus by maximum likelihood. Almost everything else is plumbing that makes the machine fast and accurate.

[1] This is the *autoregressive* factorisation. Predict each token given the prefix; multiply the per-token probabilities to get the joint. Almost every modern LLM is trained this way.

[2] Equivalently, minimise the negative log-likelihood, which (averaged over tokens) is what people report as "cross-entropy loss". Lower is better. log is natural unless someone says otherwise.

1.2 Why that humble idea is so powerful

A model that can predict "the cat sat on the _____" has, in order to do so, picked up a startling amount of structure. To put a high probability on *mat* (or *floor*, or *lap*) and a low probability on *microservice*, the model has to encode:

- the syntax of English noun phrases,
- the typical co-occurrence of cats and household objects,
- the rough fact that "sat on" takes a flat surface as its object,
- and probably a sense of register (this is a children's-book sentence, not a stack-trace).

None of that was hand-coded. It is a side effect of the maximum-likelihood objective on enough text. *Generation* is then the same model run forwards: sample a token from the distribution, append it, and repeat. That loop is the entire reason ChatGPT works.[3]

[3] The other half is instruction tuning and reinforcement learning from human feedback, which we will not cover. The pretraining objective in equation (1.1) is what builds the latent capability; RLHF shapes the surface behaviour.

1.3 What the input *actually* looks like

Equation (1.1) talks about tokens as if they were given. They are not — there is no canonical way to chop text into tokens, and the choice matters. We unpack that in Chapter 2. For now: imagine each x_t as one element of a finite *vocabulary* of size V, so the model's output at each step is a vector of V probabilities that sum to one.[4]

[4] Typical V is between $30{,}000$ and $200{,}000$ for a modern LLM. GPT-2 used 50,257; Llama 3 uses 128,000. Larger vocabularies shorten the average input but make the output projection more expensive.

1.4 What the output *actually* looks like

The model produces a vector $z_t \in \mathbb{R}^V$ of *logits* — pre-softmax scores — and turns them into a probability distribution by

$$p_\theta(x_t = v \mid x_{<t}) \;=\; \frac{\exp(z_{t,v}/\tau)}{\sum_{v'=1}^{V} \exp(z_{t,v'}/\tau)}, \tag{1.2}$$

where $\tau > 0$ is the *temperature*. At training time $\tau = 1$ and the loss is the negative log probability of the true next token. At generation time you can sample directly, take the argmax (greedy decoding), keep only the top-k most probable tokens (top-k sampling), or restrict to the smallest set whose probability exceeds p (*nucleus* or top-p sampling).[5]

[5] Lower τ sharpens the distribution and makes the model deterministic-looking. Higher τ makes it surprising, sometimes unhelpfully so. Most product systems use τ between 0.2 and 1.0 and combine it with top-$p \approx 0.9$.

1.5 Why we care about *how* p_θ is computed

The chain-rule view in equation (1.1) is architecture-free. It does not say whether $p_\theta(x_t \mid x_{<t})$ should be computed by a hash table, an n-gram count, an LSTM, or a transformer. Those are engineering choices, and they have very different cost profiles in compute and in memory. The rest of this book is about that choice — how the field arrived at the transformer, how that choice scales, and what is being proposed in 2026 to replace it.

In particular, the cost of computing $p_\theta(x_t \mid x_{<t})$ is not a fixed constant per token. It depends on *how much context* $x_{<t}$ the model is asked to attend to. For an n-gram model the cost is constant (Chapter 3). For an RNN it is also roughly constant per token, but information from far back is lost (also Chapter 3). For a dense transformer it grows *quadratically* with context length (Chapter 6). That last fact is what the second half of the book is about.

1.6 Worked example: cross-entropy on a toy vocabulary

Take $V = \{a, b, c, d\}$ and the five-token sequence $x =$ `a b a c d`. Count symbols for an empirical unigram: a:2, b:1, c:1, d:1 out of five draws, so $\hat{p}(a) = \frac{2}{5}$, $\hat{p}(b) = \hat{p}(c) = \hat{p}(d) = \frac{1}{5}$.

Model	$p(\texttt{a b a c d})$	$-\log p$ (nats)	per-token CE
(i) Uniform $p = \frac{1}{4}$ everywhere	$(\frac{1}{4})^5 = 2^{-10} \approx 0.000977$	$10 \log 2 \approx 6.93$	1.386
(ii) Unigram $\hat{p}$ above	$\frac{2}{5} \cdot \frac{1}{5} \cdot \frac{2}{5} \cdot \frac{1}{5} \cdot \frac{1}{5} \approx 3.2 \times 10^{-4}$	≈ 8.15	1.63
(iii) Bigram fit *only* on this sentence (zero elsewhere)	$1 \cdot \frac{1}{2} \cdot 1 \cdot 1 \cdot 1$	0	0

For (iii) we stored enough parameters to memorise the single training string: after seeing `a`, the next token is always `b`; after `b`, always `a`; and so on. The likelihood is maximised trivially on this one example — and would be catastrophic on any held-out text where those bigrams never appeared.[6]

Nats vs bits. "Cross-entropy" is reported in either *nats* (natural logarithm, this table) or *bits* (base-2 logarithm). One nat equals $\log_2 e \approx 1.443$ bits; divide nats by $\log 2$ to convert.

[6] This is the cartoon version of *overfitting*: the flexible model nails the training loss but does not compress structure that generalises. Real LMs smooth counts or use neural masses instead of tables, precisely to avoid this collapse.

1.7 A minimal next-token sampler (pure Python)

The softmax in equation (1.2) is the only non-linearity you need to implement decoding strategies against any oracle that returns logits.

Listing 1.1: Greedy, temperature, top-k, and nucleus sampling share one softmax routine.

```
import math
import random

def softmax(logits, temperature=1.0):
    m = max(logits)
    exps = [math.exp((x - m) / temperature) for x in
        ↪ logits]
    s = sum(exps)
    return [e / s for e in exps]

def sample_token(probs):
    r = random.random()
    c = 0.0
    for i, p in enumerate(probs):
        c += p
        if r <= c:
            return i
    return len(probs) - 1

def decode_greedy(logits_fn, prompt_ids, max_new=32):
    """Use when you want deterministic, locally optimal
        ↪ extensions."""
    out = list(prompt_ids)
    for _ in range(max_new):
        p = softmax(logits_fn(out))
        out.append(max(range(len(p)), key=lambda i: p[i])
            ↪ )
    return out

def decode_temperature(logits_fn, prompt_ids, tau=0.8,
    ↪ max_new=32):
    """tau<1 sharpens; tau>1 increases diversity."""
    out = list(prompt_ids)
    for _ in range(max_new):
        probs = softmax(logits_fn(out), temperature=tau)
        out.append(sample_token(probs))
    return out

def decode_top_k(logits_fn, prompt_ids, k=40, max_new=32)
    ↪ :
    """Truncate mass to the k largest logits - stable
        ↪ workhorse in APIs."""
    out = list(prompt_ids)
    for _ in range(max_new):
        z = logits_fn(out)
        idx = sorted(range(len(z)), key=lambda i: z[i],
            ↪ reverse=True)[:k]
        sub_z = [z[i] for i in idx]
        probs = softmax(sub_z)
```

```
        pick = sample_token(probs)
        out.append(idx[pick])
    return out

def decode_top_p(logits_fn, prompt_ids, p_mass=0.9,
    ↪ max_new=32):
    """Nucleus / top-p: renormalise over the smallest
        ↪ high-probability set (Holtzman+19)."""
    out = list(prompt_ids)
    for _ in range(max_new):
        z = logits_fn(out)
        probs = softmax(z)
        order = sorted(range(len(probs)), key=lambda i:
            ↪ probs[i], reverse=True)
        cum, keep = 0.0, []
        for i in order:
            cum += probs[i]
            keep.append(i)
            if cum >= p_mass:
                break
        sp = [probs[i] for i in keep]
        s = sum(sp)
        sp = [p / s for p in sp]
        r = random.random()
        t = 0.0
        for j, pj in enumerate(sp):
            t += pj
            if r <= t:
                out.append(keep[j])
                break
    return out
```

These functions intentionally omit batched tensor code so the control flow remains obvious; production stacks fuse softmax sampling on the GPU.

Chapter 2

Tokens — the unit of work

In Chapter 1 we wrote "each x_t is one element of a vocabulary of size V" and waved our hands. That hand wave hides one of the most consequential design choices in the entire stack. This chapter unpacks it.

2.1 Why characters and why not

The simplest possible vocabulary is the alphabet: 26 letters plus punctuation and digits. Vocab size V becomes ~ 100, the model never sees an *out-of-vocabulary* word, and tokenisation is trivial.[1]

The price you pay is sequence length. "the unfortunate consequences of quadratic scaling" is 7 words but about 50 characters. A character model sees 7× as many tokens for the same input. Since the cost of attention grows with the square of sequence length (the whole point of Part 6), that 7× becomes a 50× compute penalty per layer.

[1] The extreme of character-level modelling is byte-level: $V = 256$, every sequence of bytes is representable, no Unicode normalisation needed. Models like ByT5 take this seriously.

2.2 Why words and why not

The opposite extreme is whole words: $V \approx 10^5$ for English. Sequences become short, but you suddenly have to handle words you have never seen before — proper nouns, technical jargon, typos, every other language. Most serious work has used *subwords* since around 2016.

2.3 Subword tokenisation in one page

The dominant tokenisers all do roughly the same thing: build a vocabulary by merging frequent character pairs, then tokenise text greedily against that vocabulary.

Byte-pair encoding (BPE). Start with a vocabulary of all characters. Find the most frequent adjacent pair in your corpus (say, "`t h`"), merge it into a single token ("`th`"), and add that to the vocabulary. Repeat until you hit a target V. This was originally a compression algorithm; Sennrich et al. [10] adapted it for neural machine translation, and GPT-2 / GPT-3 popularised it for LLMs.[2]

[2] See [10] for the original NLP application and the OpenAI tokenizer notebooks for the engineering reality.

WordPiece. Used by BERT. Same merging idea, slightly different scoring criterion (likelihood gain rather than raw frequency).[3]

[3] Tied to the `bert-base-uncased` vocabulary and the famous "`##`" prefix that marks "this is a continuation of the previous token".

SentencePiece (unigram). Used by Llama, Mistral, and Gemma. Treats tokenisation as a probabilistic model: pick the segmentation that maximises a per-token probability. Operates directly on raw bytes and does not require pre-tokenising into words first, which is a real win for languages without whitespace boundaries.[4]

[4] [11]. SentencePiece is the reason Llama-style models can tokenise Japanese without a custom segmenter.

2.4 Why this affects everything downstream

A subword tokeniser fixes a vocabulary V at training time. Three consequences ripple through the rest of the system:

1. **The output projection costs $\mathcal{O}(V \cdot d)$ per token.** At each step, the model converts a hidden vector of dimension d into V logits via a linear layer. If V is 128,000 and d is 4,096 that is ~ 500 million multiply-adds per token, just for the last layer.
2. **Sequence length is in tokens, not characters.** A 1M-token context window means roughly 750k English words, or 3,000 pages of double-spaced text — assuming an English subword tokeniser. Code, Chinese, or mixed content have different token-to-character ratios.[5]
3. **The choice of tokeniser bakes in biases.** A tokeniser trained mostly on English will fragment Hindi or Korean into many small tokens, making non-English usage measurably more expensive both in compute and in context window.

[5] Rule of thumb for the OpenAI/Llama tokenisers: 1 token $\approx$ 4 English characters $\approx$ 0.75 English words. Code is denser; one short identifier can be one token.

Key idea. Tokens are a cheap-looking choice with expensive consequences. The "context length" a vendor advertises is in their tokens, not yours, and the shape of the cost curves in later chapters is in token-space.

2.5 Embeddings: turning tokens into vectors

Once you have a token id (an integer in $[0, V)$), the very first thing the model does is look it up in a table:

$$e_t \;=\; E[x_t] \in \mathbb{R}^d, \tag{2.1}$$

where $E \in \mathbb{R}^{V \times d}$ is the *embedding matrix*, learned along with everything else. Every transformer block sees only these dense vectors, not the original token ids.

For a 7B parameter model with $V = 32{,}000$ and $d = 4{,}096$ the embedding table alone is $\sim$130M parameters — roughly 2% of the model.[6]

[6] Many models tie the embedding matrix to the output projection (use the same E on the way in and the way out) which halves this cost. Tied embeddings are standard in Llama; OpenAI's are not.

2.6 The same line, three tokenisers

The following sentence (one paragraph of prose plus one line of Python) was run through three public tokenisers. The exact byte sequence is:

```
The quick brown fox jumps over the lazy dog.
def fibonacci(n): return n if n < 2 else fibonacci(n-1) +
fibonacci(n-2)
```

GPT2 (Byte-level BPE as shipped with GPT-2 — reproduced via `tiktoken`) yields 42 tokens. Representative splits include `fib`, `on`, `acci` rather than a single `fibonacci` merge:

```
The; quick; brown; fox; jumps; over; the; lazy; dog;.; \n; def; fib;
on; acci; (; n; );:; return; n; if; n; <; 2; else; fib; on; acci; …
```

BERT-BASE-UNCASED (WordPiece) yields 45 tokens. The function name splits as `fi` + `##bon` + `##ac` + `##ci` — the `##` prefix marks a continuation piece:

```
the; quick; brown; fox; jumps; over; the; lazy; dog; .; def; fi;
##bon; ##ac; ##ci; (; n; ); :; return; n; if; n; <; 2; else; fi; …
```

LLAMA-FAMILY SENTENCEPIECE (TinyLlama checkpoint — same SPM pipeline as Llama 2/3; Llama 3 weights are gated on HuggingFace so we use an open checkpoint for reproducibility) yields 45 tokens. "Fox" can split as `fo+x` depending on merge stats; `fibonacci` is still broken into several pieces because the merge never entered the vocabulary as a single token during training:

```
The; quick; brown; fo; x; j; umps; over; the; lazy; dog; .; \n; def;
fib; on; acci; (; n; );:; return; n; if; n; <; ; 2; else; fib; on;
acci; …
```

Takeaway. Code is not inherently "denser" than English — it depends on whether the BPE merges exist for your identifiers. English prose shares many merges across corpora; a rare function name is often glued together from shorter atoms unless the tokenizer saw that exact string millions of times during vocabulary construction.

2.7 Back-of-the-envelope token economics

Table 2.1 converts rough document sizes into token counts with the OpenAI `cl100k_base` convention ($\approx$ 1.3 tokens per English word for mixed prose). Dollar amounts use an *illustrative* GPT-class input price of USD 2.50 per million tokens — swap in your provider's current page when you budget.[7]

[7] Pricing moves quarterly; the point is order-of-magnitude, not the third decimal.

Table 2.1: Rule-of-thumb token loads for budgeting API spend (not for fitting KV caches).

Document	Approx. tokens	Minutes of GPU "talk time" *	Illustr. USD @ $2.50/M
10k-word English essay	$\sim$13,000	—	$\sim$\$0.03
10k-word Hindi essay	$\sim$28,000–35,000	—	$\sim$\$0.07–\$0.09
1k-line Python file ($\sim$40 char/line)	$\sim$30,000–45,000	—	$\sim$\$0.08–\$0.11
100-page contract ($\sim$450 words/page)	$\sim$60,000–75,000	—	$\sim$\$0.15–$\sim$\$0.19

*GPU time is orthogonal to dollar pricing but scales with tokens $\times$ depth.

Non-English text routinely consumes *more* tokens per unit of meaning when the tokenizer was trained primarily on English — so equality-sensitive products should quote prices *per language* or risk silently taxing non-English users.[8]

[8] See also fragmentation studies on multilingual `cl100k` and Llama tokenisers; the qualitative point is stable even when exact factors shift.

The next chapter is about how, before the transformer, people stitched these embeddings together over time without an attention matrix at all.

Chapter 3

The pre-transformer era

The transformer is a 2017 paper. Language modelling has been a serious research topic for at least 70 years before that. Most of the intuitions in modern LLMs — the chain-rule decomposition, the importance of context, the trade-off between fixed and unbounded memory — are not new. They have been re-discovered, with increasing computational firepower, several times.

This chapter is a tour of what came before the transformer, framed in the language we will use in the rest of the book. It is short, but it matters: many of the "new" subquadratic architectures in Part V are direct descendants of pre-transformer ideas that the field stopped using because GPUs got big enough to brute-force the problem.

3.1 n-gram models — counting your way to fluency

The first practical language models were just *count tables*. Pick a window size k (usually 3 to 5). For every k-tuple of tokens you ever saw in a training corpus, store

$$p(x_t \mid x_{t-k+1}, \ldots, x_{t-1}) \;=\; \frac{\#(x_{t-k+1}, \ldots, x_{t-1}, x_t)}{\#(x_{t-k+1}, \ldots, x_{t-1})}. \tag{3.1}$$

That is the entire model.[1]

Strengths. O(1) compute per token. Embarrassingly parallel to train. For $k = 5$, competitive with much fancier models on speech-recognition language-model rescoring up to roughly 2010.[2]

Weaknesses. Fundamentally bounded by k. An n-gram model literally cannot use information from more than $k - 1$ tokens back, even in principle. The state space

[1] In practice you smooth the counts so unseen tuples get a tiny non-zero probability — Kneser-Ney smoothing [1] was the dominant technique for two decades.

[2] Google's 2007 paper described training 5-grams over 2 trillion words for machine translation, several years before any neural alternative was viable [2].

grows as V^k, so going from 5-grams to 10-grams blows up by twelve orders of magnitude.

That hard bound — *context up to k tokens, period* — is the original sin every architecture since has been trying to escape.

3.2 Recurrent neural networks (RNNs)

A recurrent network throws the count table out and replaces it with a single hidden vector $h_t \in \mathbb{R}^d$ that is updated step by step:

$$h_t \;=\; \tanh(W_h h_{t-1} + W_x e_t + b), \quad p_\theta(x_{t+1} \mid x_{\le t}) \;=\; \mathrm{softmax}(W_o h_t). \tag{3.2}$$

Now the model can in principle use *all* prior tokens, because each h_t is a function of the entire prefix.[3] Compute per token is $\mathcal{O}(d^2)$, independent of t, which is a beautiful property. We will return to that.

[3] The clean derivation goes back to [3]; [4] put it on the LM map for modern NLP.

The vanishing gradient problem. The catch is that "in principle" is doing all the work. Training the RNN by backpropagation through time multiplies many Jacobians together; with the tanh non-linearity, the gradients vanish (or, worse, explode) within roughly 20 steps. Vanilla RNNs effectively have a context of about that length, and nobody could train them stably enough to do better.

3.3 LSTMs and GRUs — the gated decade

The fix that worked is *gating*. A long short-term memory cell [5] replaces the simple update in equation (3.2) with a small machine made of *gates*:

$$f_t = \sigma(W_f[h_{t-1}, e_t] + b_f) \qquad \text{forget gate} \tag{3.3}$$

$$i_t = \sigma(W_i[h_{t-1}, e_t] + b_i) \qquad \text{input gate} \tag{3.4}$$

$$o_t = \sigma(W_o[h_{t-1}, e_t] + b_o) \qquad \text{output gate} \tag{3.5}$$

$$\tilde{c}_t = \tanh(W_c[h_{t-1}, e_t] + b_c) \qquad \text{candidate} \tag{3.6}$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \qquad \text{cell state} \tag{3.7}$$

$$h_t = o_t \odot \tanh(c_t). \tag{3.8}$$

The cell state c_t flows along an additive path, so its gradient does not vanish; the gates decide what to write, what to keep, and what to read. Cho et al. [6] simplified this to the GRU, which has fewer parameters and behaves similarly in practice.[4]

LSTMs ran the world from roughly 2014 to 2017. Google Translate, Siri's on-device speech, the original Alexa wake-word detector — all LSTM under the hood.

[4] If you have ever wondered why *Mamba* (Chapter 13) keeps using the words "selective", "state", and "gate" — it is because Mamba is the same idea, polished and re-discovered with hardware-aware tricks.

Why LSTMs eventually lost. Two reasons.

1. **Sequential by construction.** Each h_t depends on h_{t-1}. You cannot parallelise across the time axis; the GPU sits half idle. Transformers, in contrast, evaluate every position simultaneously during training.
2. **Compressed memory.** Everything from the past has to be squeezed into a single h_t vector of fixed size. With enough capacity an LSTM can remember a few thousand tokens; beyond that, things blur. We will see this exact trade-off come back as a feature of state-space models, where it is sometimes a virtue.

3.4 Encoder-decoder, sequence-to-sequence, and the original "attention"

In 2014, two papers [6, 7] proposed *sequence-to-sequence* models for machine translation: an encoder LSTM reads the source sentence into a fixed-size vector, a decoder LSTM generates the target one token at a time, conditioned on that vector.

This worked for short sentences and fell over for long ones — the bottleneck of compressing an entire paragraph into one vector was too tight.

The fix is the original meaning of the word *attention*. Bahdanau et al. [8] let the decoder, at every step, look back at *all* the encoder hidden states and produce a weighted average:

$$c_t = \sum_{i=1}^{n} \alpha_{t,i}\, h_i^{\text{enc}}, \qquad \alpha_{t,i} = \frac{\exp(\text{score}(h_t^{\text{dec}}, h_i^{\text{enc}}))}{\sum_j \exp(\text{score}(h_t^{\text{dec}}, h_j^{\text{enc}}))}. \tag{3.9}$$

The weights $\alpha_{t,i}$ say "how much should the decoder pay attention to source position i when emitting target token t". They are softmax normalised. They are computed from the current decoder state and the encoder states.

Key idea. The attention layer that runs every transformer in 2026 is a direct descendant of the Bahdanau attention bolted onto an LSTM in 2014. The transformer's contribution is to delete the LSTM and use *only* attention.

3.5 The 2017 turn

What Vaswani et al. [9] pointed out is that you can build a sequence model out of attention alone. No recurrence, no convolution. Stack attention layers, mix in some position-wise feed-forward layers, add positional encodings so the model knows token order, and you get a model that:

- parallelises across the time axis (every position is computed at once),

- lets every token attend to every other token in a single layer (no vanishing gradient over distance),
- and trains roughly an order of magnitude faster on the same hardware.

The price is a quadratic dependence on sequence length. That price is what the rest of the book is about.

The 2017 paper was about machine translation, but within four years the same recipe — scaled up by three orders of magnitude — was producing GPT-3, a language model that could do arithmetic, write code, and pass standardised exams. The transformer ate the field. Part II is the careful explanation of why.

Listing 3.1: Vanilla RNN over time (matches equation 3.2, one step at a time).

```
import torch

def rnn_forward(e, W_h, W_x, b_h, W_o, b_o, h0=None):
    # e: [T, B, d_in] token embeddings; W_h: [h, h]; W_x:
        ↪ [d_in, h]; b_h: [h]
    # W_o: [h, V]; b_o: [V]; returns logits per step [T,
        ↪ B, V] and final h [B, h]
    T, B, d_in = e.shape
    h_dim = W_h.shape[0]
    h = torch.zeros(B, h_dim, device=e.device, dtype=e.
        ↪ dtype) if h0 is None else h0
    logits_steps = []
    for t in range(T):
        # [B,h] = tanh([B,h]@[h,h] + [B,d_in]@[d_in,h] +
            ↪ [h])
        h = torch.tanh(h @ W_h + e[t] @ W_x + b_h)   #
            ↪ hidden state
        logits_steps.append(h @ W_o + b_o)          # [B,
            ↪ V] softmax-ready logits
    return torch.stack(logits_steps, dim=0), h
```

This is faithful to the math but forces a Python loop over T: the GPU cannot fuse the time axis, which is why seq2seq training on LSTMs never matched transformer throughput once attention could be parallelised across positions (Chapter 4).

Listing 3.2: One LSTM step with explicit gates (same layout as the equations above).

```
import torch

def lstm_cell(h_prev, c_prev, e_t, W_i, W_f, W_o, W_c,
    ↪ b_i, b_f, b_o, b_c):
    # h_prev: [B, h]; c_prev: [B, h]; e_t: [B, d]; each
        ↪ W_* : [d + h, h]
    x = torch.cat([h_prev, e_t], dim=1)            # [B, d
        ↪ +h] - concat for one linear
    i = torch.sigmoid(x @ W_i + b_i)               # input
        ↪ gate
```

```
    f = torch.sigmoid(x @ W_f + b_f)             # forget
        ↪ gate
    o = torch.sigmoid(x @ W_o + b_o)             # output
        ↪ gate
    c_tilde = torch.tanh(x @ W_c + b_c)            #
        ↪ candidate cell content
    c = f * c_prev + i * c_tilde                   # cell
        ↪ state (additive path)
    h = o * torch.tanh(c)                          #
        ↪ hidden output
    return h, c
```

Mapping to the prose derivation: `f` is forget, `i` is input, `o` is output, and the line updating `c` is the cell update; `c_tilde` is $\tilde{c}_t$.

Parameter count for one LSTM cell. Each gate is an affine map $\mathbb{R}^{d+h} \to \mathbb{R}^h$: matrix $(d+h) \times h$ and bias h, i.e. $(d+h)h + h = dh + h^2 + h$ parameters. Four gates give

$$4\,(dh + h^2 + h). \tag{3.10}$$

With $d = h = 1024$ this is $4 \cdot (2 \cdot 1024^2 + 1024) \approx 8.39$ million trainable parameters *per cell*. A depth-L stacked LSTM multiplies that by L.

For comparison, a transformer block at hidden width d carries attention plus feed-forward weights scaling as $\mathcal{O}(d^2)$ with a much larger coefficient (multiple matrices of size $d \times d$ plus the FFN upswing to $4d^2$-ish in the usual GeLU configuration). Chapter 4 works the accounting for a full block; the qualitative point here is that a *single* LSTM cell is compact, but depth-L recurrence cannot escape sequential steps at runtime, whereas a transformer layer pays more parameters per layer yet trains with full time-parallelism.

Part II

The Transformer

Chapter 4

The transformer recipe

This chapter walks through a standard decoder-only transformer block end to end, the way you would actually code it. The next chapter zooms into the attention sub-layer; the chapter after that derives why that sub-layer is the cost driver.

4.1 The block, in one picture

A decoder-only transformer is a stack of L identical blocks. Each block takes a tensor of shape $[n, d]$ — n tokens, each represented by a vector of $d_{\text{model}} = d$ dimensions — and returns a tensor of the same shape. Inside the block, two sub-layers do the work:

1. *Self-attention*, which lets every token mix information from every other (earlier) token.
2. A *position-wise feed-forward network* (a small MLP applied independently to each token), which gives the model nonlinear capacity beyond what attention alone provides.

Each sub-layer is wrapped in a *residual connection* and a *normalisation* layer:

$$z \leftarrow x + \text{Attn}(\text{LN}(x)) \tag{4.1}$$

$$y \leftarrow z + \text{MLP}(\text{LN}(z)). \tag{4.2}$$

That is a complete block.[1] The whole model wraps token embeddings + positional information as input, runs L of these blocks, applies a final LN, and projects to logits over the vocabulary.

[1] Two flavours: *post-LN* (the original 2017 paper) puts the LN *after* the residual sum; *pre-LN* (now standard) puts it *before*, as written here. Pre-LN trains more stably at depth.

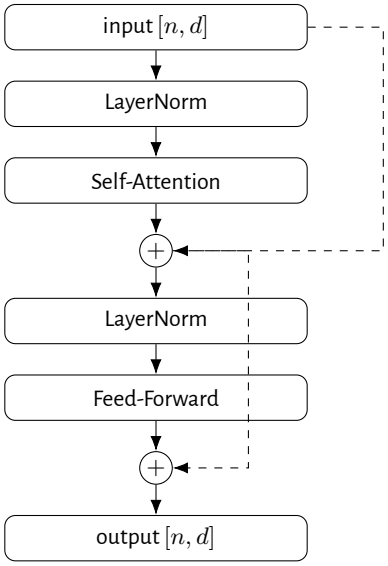

Figure 4.1: One transformer block (pre-LN). Dashed lines are residual connections; the two solid sub-blocks (attention and feed-forward) are where all the actual work happens.

4.2 The pieces, named

Embeddings. Token ids $\to$ vectors of dimension d via the table from Chapter 2.

Positional information. Attention is permutation-invariant — if you shuffle the tokens, the output is the same. Real language is not. The fix is to add positional information to the embeddings.[2]

Self-attention. The hero of Chapter 5. Lets every token's representation be a weighted average of every other token's value, where the weights are computed from the dot product of the current token's *query* with the others' *keys*.

Feed-forward (MLP). A two-layer MLP applied independently at each position:

$$\text{MLP}(x) \;=\; W_2\,\sigma(W_1 x + b_1) + b_2, \tag{4.3}$$

with hidden dimension d_{ff}, typically $4d$ (so 16,384 for $d = 4{,}096$). σ is GeLU, SwiGLU, or similar. This is where most of the parameters live — the FFN is roughly $8d^2$ parameters per layer, vs. $4d^2$ for attention's projection matrices.[3]

LayerNorm / RMSNorm. A normalisation that rescales each token's hidden vector to have zero mean and unit variance (LN) or just unit norm (RMSNorm). Standard since 2018, plus a small learned scale and shift.

Output projection. A linear layer of shape $[d, V]$ followed by a softmax (equation 1.2) to produce next-token probabilities.

[2] Three families have dominated. (i) Learned absolute positions (BERT, GPT-2): add a learned vector per index. (ii) Sinusoidal (Vaswani 2017): use hand-crafted sine waves at geometric frequencies. (iii) RoPE (rotary position embeddings, 12): apply a position-dependent rotation inside the attention's Q and K, which is the de-facto modern choice (Llama, Mistral, Gemma). RoPE composes cleanly with extrapolation tricks like YaRN [13].

[3] For a 7B model with $d = 4096$, $L = 32$: attention $\approx 32 \cdot 4d^2 \approx 2.1$B params, FFN $\approx 32 \cdot 8d^2 \approx 4.3$B params, plus embeddings and norms. The FFN dominates the parameter count even though attention dominates the asymptotic compute.

4.3 Why the transformer beat the LSTM

Three reasons matter.

1. **Parallelism on the time axis.** Equation (3.2) forced h_t to be computed before h_{t+1}. Self-attention computes all positions at once via a single matrix multiply; the GPU stays busy.
2. **No vanishing gradient over distance.** An LSTM's signal from token 1 to token 1000 has to survive 999 multiplicative updates. In a transformer there is a direct dot product between any two positions in any layer.
3. **Easy depth scaling.** Residuals + LayerNorm let you stack 100+ identical blocks with no special handling. RNNs above ~ 4 layers were notoriously hard to train.

4.4 What it cost

The transformer pays for those three wins with one bill: every layer's attention sub-block computes an $n \times n$ matrix of pairwise similarities between tokens. Compute scales as n^2. Memory for the attention matrix scales as n^2. The KV cache used at generation time scales as n per layer, but with multiplicatively large constants.

For n of a few thousand — the regime of GPT-2 and GPT-3 — this was fine. For n of one million — what 2026's agentic workloads want — it is the defining problem of the field. Chapter 6 derives the n^2 cost carefully; Part IV documents the engineering response; Part V covers the architectures that try to escape it.

Listing 4.1: Pre-LN decoder block matching Figure 4.1 and equation 4.2.

```
import math
import torch
import torch.nn as nn
import torch.nn.functional as F

class CausalSelfAttention(nn.Module):
    # x: [B, T, d] -> same; one softmax attention over
        ↪ past tokens (causal).
    def __init__(self, d_model: int, n_heads: int):
        super().__init__()
        assert d_model % n_heads == 0
        self.d_model, self.n_heads = d_model, n_heads
        self.d_head = d_model // n_heads
        self.q_proj = nn.Linear(d_model, d_model)   # [B,
            ↪ T,d]@[d,d]
        self.k_proj = nn.Linear(d_model, d_model)
        self.v_proj = nn.Linear(d_model, d_model)
        self.o_proj = nn.Linear(d_model, d_model)
```

```
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        B, T, d = x.shape
        q = self.q_proj(x).view(B, T, self.n_heads, self.
            ↪ d_head).transpose(1, 2)
        k = self.k_proj(x).view(B, T, self.n_heads, self.
            ↪ d_head).transpose(1, 2)
        v = self.v_proj(x).view(B, T, self.n_heads, self.
            ↪ d_head).transpose(1, 2)
        scores = (q @ k.transpose(-2, -1)) / math.sqrt(
            ↪ self.d_head)  # [B,nh,T,T]
        mask = torch.triu(torch.ones(T, T, device=x.
            ↪ device), diagonal=1).bool()
        scores = scores.masked_fill(mask, float("-inf"))
        attn = F.softmax(scores, dim=-1)
        y = attn @ v
            ↪                                              #
            ↪ [B,nh,T,dh]
        y = y.transpose(1, 2).contiguous().view(B, T, d)
            ↪       # [B,T,d]
        return self.o_proj(y)

class TransformerBlock(nn.Module):
    def __init__(self, d_model: int, n_heads: int, d_ff:
        ↪ int):
        super().__init__()
        self.ln1 = nn.LayerNorm(d_model)
        self.attn = CausalSelfAttention(d_model, n_heads)
        self.ln2 = nn.LayerNorm(d_model)
        self.fc1 = nn.Linear(d_model, d_ff)
        self.fc2 = nn.Linear(d_ff, d_model)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        h = self.ln1(x)
        h = self.attn(h)
        x = x + h
        h = self.ln2(x)
        h = self.fc2(F.gelu(self.fc1(h)))
        x = x + h
        return x
```

`LayerNorm` precedes each sub-layer; the residual adds skip connections around attention and around the two-layer FFN, matching equation 4.2. `CausalSelfAttention` instantiates the $Q/K/V$ projections and head-wise QK^{T} softmax map described in prose; the causal mask encodes decoder-only masking over sequence length T.

The largest slices are the SwiGLU triple ($\approx$ 176M parameters per layer) and the untied embedding plus LM head ($\approx$ 1.05B combined). If embeddings were *tied* with the output projection, one would delete one $V \cdot d$ matrix and reuse the input embedding table for logits, saving roughly half a billion parameters while forcing

Component	Formula (Llama-3 8B symbols)	Parameters
Token embeddings	$V \cdot d$	$128{,}256 \cdot 4{,}096 \approx 0.526$B
Per-layer attention	$2d^2$ (Q, O) $+\ 2\,d\,(n_{\text{kv}} d_{\text{head}})$ (K, V)	$\approx 4.19 \times 10^7$
Per-layer SwiGLU FFN	$3\,d\,d_{\text{ff}}$	$3 \cdot 4{,}096 \cdot 14{,}336 \approx 1.76 \times 10^8$
Per-layer RMSNorm	$2d$ (input to attn + FFN)	$8{,}192$ (negligible)
All $L = 32$ layers	$32 \times (\text{attn} + \text{ffn} + \text{norm})$	≈ 6.98B
LM head (untied)	$V \cdot d$	≈ 0.526B
Final norm	d	$4{,}096$
Total (rounded)		$\approx$ **8.03**B

Table 4.1: Parameter accounting for `Meta-Llama-3-8B`-style shapes: $d = 4{,}096$, $L = 32$, $H = 32$, $n_{\text{kv}} = 8$, $d_{\text{ff}} = 14{,}336$, $V = 128{,}256$, SwiGLU MLP, separate embedding and LM head matrices.

logits to remain linear functions of the same rows used at the input.

RoPE in one breath. Split each q and k into $d_{\text{head}}/2$ plane pairs (q_{2k}, q_{2k+1}). Position m rotates that pair by angle $m\theta_k$ with frequencies θ_k spaced geometrically. Dot products $q_i^{\text{rot}} \cdot k_j^{\text{rot}}$ then depend on q, k only through the *difference* $j - i$ because rotations compose additively in angle — exactly the inductive bias decoder-only LMs want. That is why RoPE is the default in Llama, Mistral, and Gemma. YaRN [13] rescales those frequencies (or interpolates bases) so the same checkpoints extrapolate to longer contexts without retraining from scratch.

Chapter 5

Self-attention from scratch

If the transformer block in Chapter 4 has a single piece worth understanding cell-by-cell, it is the self-attention sub-layer. This chapter writes it out, with all the matrix shapes.

5.1 The setup

Take the input to one block: a tensor $X \in \mathbb{R}^{n \times d}$, where n is the number of tokens and $d = d_{\text{model}}$ is the hidden dimension. Each row x_t is the current representation of token t.

We are going to produce an output of the same shape: a new representation for each token that has incorporated information from earlier tokens.

5.2 Queries, keys, and values

Project the input into three different spaces:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V, \tag{5.1}$$

with three learned matrices $W_Q, W_K, W_V \in \mathbb{R}^{d \times d_{\text{head}}}$. Each of Q, K, V is $[n, d_{\text{head}}]$.[1]

[1] The intuition. *Query* = "what am I looking for?". *Key* = "what do I advertise as being about?". *Value* = "what do I actually carry that you'd want to read?". The attention weight from token t to token s is high when t's query and s's key align; the value v_s is what gets copied into t's next representation.

5.3 The attention matrix

Form pairwise scores via dot product, scale by $\sqrt{d_{\text{head}}}$ so the softmax does not saturate, and softmax row-wise:

$$A \;=\; \text{softmax}\!\left(\frac{QK^{\top}}{\sqrt{d_{\text{head}}}}\right) \;\in\; \mathbb{R}^{n\times n}. \tag{5.2}$$

$A_{t,s}$ is "how much should token t attend to token s". The softmax is taken over rows, so each row sums to one — every token spreads its attention budget over all other tokens.[2]

[2] The scaling $1/\sqrt{d_{\text{head}}}$ is not a cosmetic choice. Without it, dot products grow with $\sqrt{d_{\text{head}}}$ in expectation, the softmax saturates, and gradients vanish. Vaswani et al. [9] document this in their section 3.2.1.

5.4 The output

Use A as a row-stochastic matrix to mix values:

$$H \;=\; AV \;\in\; \mathbb{R}^{n\times d_{\text{head}}}. \tag{5.3}$$

Row t of H is the weighted sum $\sum_s A_{t,s} v_s$ of all values, weighted by how much token t wanted to attend to token s.

Key idea. Self-attention is one matrix multiply ($QK^{\top}$), one row-wise softmax, one more matrix multiply (AV). The three steps look small. The middle matrix A is $n \times n$. That is the entire story of this book.

5.5 Multi-head attention

The single-head version has limited capacity: every position attends to every other through a single dot-product space of dimension d_{head}. In practice, you run H of these in parallel:

$$H_h \;=\; \text{softmax}\!\left(\frac{Q_h K_h^{\top}}{\sqrt{d_{\text{head}}}}\right) V_h, \quad h = 1, \dots, H, \tag{5.4}$$

with $d_{\text{head}} = d/H$. Concatenate the head outputs along the feature axis and apply a final output projection $W_O \in \mathbb{R}^{d\times d}$:

$$\text{MHA}(X) \;=\; [H_1, H_2, \dots, H_H]\, W_O \;\in\; \mathbb{R}^{n\times d}. \tag{5.5}$$

Different heads can specialise — one might track syntax, another co-reference, another long-range topical similarity. The cost adds linearly in H because each head shrinks d_{head} proportionally.[3]

[3] Concretely, for $d = 4096$ and $H = 32$, each head has $d_{\text{head}} = 128$. This is a common choice; d_{head} between 64 and 128 has been the sweet spot since GPT-3.

5.6 The causal mask

For an *autoregressive* language model, token t is not allowed to look at tokens $> t$ — that would be cheating during training (the model could just copy the next token from the input). The fix is the *causal mask*: before the softmax, set every entry above the diagonal of $QK^\top$ to $-\infty$. The softmax of $-\infty$ is zero, so $A_{t,s} = 0$ whenever $s > t$. The lower triangular structure is what the entire generation loop depends on.

5.7 Why self-attention is genuinely different from an LSTM

In an LSTM, information from token s reaches token t (with $s \ll t$) through a chain of $t-s$ updates of the hidden state. Each update can distort, drop, or blur the signal.

In self-attention, the contribution of token s to token t is a single dot product followed by a softmax-weighted sum. There is no chain. Distance in the sequence does not, by itself, attenuate the signal.

Key idea. A transformer can "copy" a fact from token 1 into token 1000 in a single attention layer. An LSTM has to relay it through 999 hidden-state updates. This is why transformers are so much better at long-range dependencies — and why the resulting compute bill is paid in an $n \times n$ matrix.

5.8 The shapes, summarised

For a transformer with n tokens, $d_{\text{model}} = d$, H heads, and $d_{\text{head}} = d/H$:

Tensor	Shape	Notes
X	$[n, d]$	input to the block
Q, K, V per head	$[n, d_{\text{head}}]$	queries, keys, values
$QK^\top$ per head	$[n, n]$	*the* matrix
A per head	$[n, n]$	after row-wise softmax
AV per head	$[n, d_{\text{head}}]$	weighted values
concat	$[n, d]$	all heads stitched
output	$[n, d]$	after W_O

The bold row to remember is $[n, n]$. One $n \times n$ tensor per head per layer. For $n = 10^6$ and 32 heads and 32 layers, that is $32 \cdot 32 \cdot (10^6)^2 = 10^{15}$ cells per forward pass. Even at one byte per cell that is a petabyte of intermediate state. The next chapter spells out the consequences.

Listing 5.1: Single-head scaled dot-product attention (equation 5.2), causal LM variant.

```
import math
import torch
import torch.nn.functional as F

def sdpa_single_head(x, W_q, W_k, W_v, causal=True):
    # x: [n, d]; W_q,W_k,W_v: [d, d_head] -> output [n,
        ↪ d_head]
    n, d = x.shape
    d_head = W_q.shape[1]
    q = x @ W_q                          # [n, d_head]
    k = x @ W_k                          # [n, d_head]
    v = x @ W_v                          # [n, d_head]
    scores = (q @ k.T) / math.sqrt(d_head)    # [n, n]
    if causal:
        mask = torch.triu(torch.ones(n, n, dtype=torch.
            ↪ bool), diagonal=1)
        assert mask.shape == (n, n)
        scores = scores.masked_fill(mask, float("-inf"))
    attn = F.softmax(scores, dim=-1)          # row-wise;
        ↪ rows sum to 1
    return attn @ v                            # [n,
        ↪ d_head]
```

This is exactly "QK^{T}, mask, softmax, multiply by V" with no fused kernels; production training stacks heads and batches B leading axes.

What does an attention head actually learn? Interpretability work on small transformers shows heads are not interchangeable: many behave like fixed positional routes or shallow circuits rather than generic similarity.[4] A canonical decoder example is an *induction head*: when the context repeats a pattern `A B … A`, this head attends from the second `A` back to `B` so the model can predict the token that followed the first `A` — supporting in-context copying and small algorithmic steps. The attention matrix is sparse and structured (previous-token + source match), not a diffuse softmax over everything. Multi-head attention exists precisely so several such low-rank circuits can coexist: some heads gate syntax-like agreement, others implement retrieval-like jumps, others approximate positional offsets. Width d split into H heads trades parameter efficiency for a *bank* of parallel routing patterns.

[4] See *e.g.* "What Does BERT Look At?" [46] for encoder self-attention and Olsson et al. on *induction heads* in small LMs.

Chapter 6

Why dense attention is unavoidably $\mathcal{O}(n^2)$

SELF-ATTENTION from Chapter 5 forms an $n \times n$ matrix per head per layer. This chapter is the careful accounting of what that costs in compute, in memory, and in dollars at the context lengths people actually want.

6.1 The compute count

For one attention head with n tokens and head dimension d_{head}:

- Forming Q, K, V costs $3 \cdot n \cdot d \cdot d_{\text{head}}$ FLOPs (three projection matmuls). Linear in n.
- Forming $QK^\top$ costs $n^2 \cdot d_{\text{head}}$ FLOPs. *Quadratic in* n.
- The softmax costs $\mathcal{O}(n^2)$ operations.
- Forming AV costs $n^2 \cdot d_{\text{head}}$ FLOPs. Quadratic again.
- The output projection costs $n \cdot d^2$. Linear.

Summing across H heads and L layers, the dominant term is $\mathcal{O}(L \cdot H \cdot n^2 \cdot d_{\text{head}}) = \mathcal{O}(L \cdot n^2 \cdot d)$.

Key idea. Attention's compute scales as $\mathcal{O}(n^2 d)$ per layer. Doubling n does not double the work — it *quadruples* it. Going from 32K to 1M tokens (a factor of ~ 32) makes attention $\sim 1024\times$ more expensive per layer.

6.2 The memory count

Two kinds of memory bills come due.

Activation memory during a forward pass. The attention matrix A has n^2 entries per head per layer. For $n = 10^6$, one head, fp16: $2 \cdot 10^{12}$ bytes $= 2$ TB.[1]

[1] Per layer. Per head. Even with bf16 or fp8 we are still in the multi-gigabyte range per head per layer at $n = 10^6$. This is why FlashAttention, which never *materialises* A in main memory, was such a big deal — Chapter 10.

KV cache memory at inference. During autoregressive generation, to answer "what comes after token t" the model needs the keys and values for tokens $1, \ldots, t$. It would be wasteful to recompute them at every step, so they are cached. The cache size grows linearly in t:

$$\text{KV bytes per token} \;=\; 2 \cdot L \cdot H \cdot d_{\text{head}} \cdot s, \tag{6.1}$$

where s is the bytes per scalar (2 for fp16). For a 7B-class model with $L = 32$, $H = 32$, $d_{\text{head}} = 128$, fp16: $\sim 524{,}288$ bytes $\approx$ 0.5 MB *per token*.[2]

[2] The arithmetic: $2 \cdot 32 \cdot 32 \cdot 128 \cdot 2 = 524{,}288$. One H100 has 80 GB of HBM. Divide: $\sim 160{,}000$ tokens before the KV cache fills the entire card. Frontier 70B-class models are roughly 2–4$\times$ worse per token.

1M tokens hits the wall. 0.5 MB/token $\times\ 10^6$ tokens = 524 GB of KV cache. The biggest single GPU available in 2026 is roughly 192 GB. A 1M-token transformer cannot keep its own working memory on a single device, full stop. This is the engineering motivation for everything in Part IV (FlashAttention, MQA, GQA, paged caches) and the architectural motivation for everything in Part V (state-space models, sparse attention, hybrids).

6.3 Translated into dollars

Run a 70B model with dense attention at $n = 10^6$ on a cluster of H100s. Even with FlashAttention-3 reducing constants, an order-of-magnitude estimate puts a single forward pass on the order of tens of GPU-seconds. At cloud prices, that is dollars *per inference*, before counting the fact that the cache will not fit on one machine and so multi-GPU communication adds another layer of slowdown.

This is what people mean when they say "transformers don't scale to long context". The math works on paper. The hardware bill does not work on a business model.

Key idea. The engineering question for 2026 is not "how do we make a model with a 1M token context window", it is "how do we make a 1M token context *cheap enough to serve to actual users at scale*".

6.4 Why this isn't the dimensions' fault

Notice what we did *not* count as quadratic: anything in d. The hidden dimension d shows up linearly. The vocabulary V shows up linearly in the embedding/output layers. The number of layers L shows up linearly. *Only* sequence length n is quadratic.

That asymmetry is structural. Each token has to look at every other token because attention is, by definition, an all-pairs operation. If you want to remove the n^2, you have to change *which pairs* get scored. That is the topic of Part V.

6.5 Bridge to scaling laws

Before we discuss how to fix the n^2, we have to address a different question: even at *short* context, how does the loss of a transformer behave as we make it bigger? The 2020 paper that answered that question [14] is the reason any of this matters at all — it turned LM training from research into engineering. Part III is about that paper and its 2022 correction.

Listing 6.1: KV-cache footprint (FP16); multiply by batch size if distinct sequences each carry their own cache.

```
def kv_cache_bytes_mha(n, L, H, d_head, dtype_bytes=2):
    """Multi-head attention: store full K,V per head."""
    return L * n * 2 * H * d_head * dtype_bytes

def kv_cache_bytes_gqa(n, L, n_kv, d_head, dtype_bytes=2)
    ↪ :
    """Grouped-query attention: only n_kv key/value heads
        ↪  are materialised."""
    return L * n * 2 * n_kv * d_head * dtype_bytes

def kv_cache_bytes_mqa(n, L, d_head, dtype_bytes=2):
    return kv_cache_bytes_gqa(n, L, 1, d_head,
        ↪ dtype_bytes)
```

n (tokens)	MHA (GiB)	GQA-8 (GiB)	MQA (GiB)
8,192	4.0	1.0	0.13
131,072	64	16	2.0
10^6	488	122	15.3
10M (10^7)	4,883	1,221	153

Table 6.1: Llama-3 8B shapes (L=32, H=32, d_{head}=128); MHA stores all H KV heads; GQA stores n_{kv}=8. Which sequence length (among these rows) first pushes KV alone past one 80 GiB H100? MHA and GQA do so at n=10^6 (488 GiB and 122 GiB); MQA still fits at n=10^6 and only approaches $\sim$150 GiB at n=10^7.

The closed-form expressions above agree with the three `kv_cache_bytes_*` helpers (modulo rounding to table digits).

Dollars per long-context forward (order of magnitude). Take a Llama-3 70B-class dense decoder with $d_{\text{model}} \approx 8192$, $L = 80$, causal attention, and a forward pass over the full prompt (prefill). The quadratic attention terms contribute very roughly $4Ln^2 d_{\text{model}}$ multiply-adds—plugging $n = 128 \cdot 1024$ yields about 4.5×10^{16} FLOPs (≈45 PFLOPs) from the $QK^\top$ and softmax$(QK^\top)V$ contractions alone, before FFN matmuls. An H100-class GPU sustains perhaps $\mathcal{O}(10^2)$ TFLOP/s on dense bf16 attention with FlashAttention-2; treating ≈400 TFLOP/s as an optimistic effective rate gives ≈100–150 GPU-s (≈0.03–0.04 GPU-h) for that attention slab alone at 128k—real stacks spend extra time on FFNs, memory bandwidth, and kernels, so full-wall-clock prefill is often higher.

Scaling to $n = 10^6$ grows the quadratic term by $\approx (10^6/1.3 \times 10^5)^2 \approx 58\times$, pushing the same accounting toward multi-hour GPU territory unless batching, clustering, or sparse/subquadratic layers intervene. Using illustrative on-demand pricing of $\$\,3$ per GPU-hour (exact vendor quotes vary), even fractions of a GPU-hour per multi-million-token prefill quickly imply dollars to tens of dollars *per query class* at long context, which is why Part V treats n^2 as an architectural problem, not only an implementation one.

Part III

Scaling laws — when bigger really is better

Chapter 7

Scaling laws — Kaplan et al.

Most of what we now call "the LLM playbook" rests on a single empirical observation made between 2017 and 2020 and crystallised by Kaplan et al. [14]. The observation: as you make a transformer language model bigger, give it more data, and train it longer, its loss follows a smooth power law in each of those resources. Plotting loss against any of them on log-log axes gives a *straight line*.

That line is the reason the field collectively decided to spend hundreds of millions of dollars per training run. Before it, scaling up was speculative. After it, scaling up was a quantitative bet with predictable payoff.

7.1 The three power laws

Kaplan et al. [14] swept seven orders of magnitude of model size and five of data, training transformers on text. They fit:

$$L(N) \approx \left(\frac{N_c}{N}\right)^{\alpha_N}, \qquad \alpha_N \approx 0.076 \tag{7.1}$$

$$L(D) \approx \left(\frac{D_c}{D}\right)^{\alpha_D}, \qquad \alpha_D \approx 0.095 \tag{7.2}$$

$$L(C) \approx \left(\frac{C_c}{C}\right)^{\alpha_C}, \qquad \alpha_C \approx 0.050, \tag{7.3}$$

where N is non-embedding parameters, D is dataset tokens, and C is training compute in PF-days.[1] Each holds when the named resource is the *bottleneck*; when it is not, the loss flattens out into a regime governed by whatever is the next-tightest constraint.

[1] The constants N_c, D_c, C_c and exponents α_* are fitted from the empirical sweeps; they shift slightly with corpus and architecture but the shape — a power law over many orders of magnitude — is remarkably robust.

7.2 What "straight on log-log" actually means

Take logs of $L(N) = (N_c/N)^{\alpha_N}$:

$$\log L \;=\; \alpha_N \log N_c - \alpha_N \log N. \tag{7.4}$$

Plot $\log L$ against $\log N$: a line with slope $-\alpha_N$. For $\alpha_N \;=\; 0.076$, every $10\times$ in N buys roughly an 0.076-decade fall in loss, i.e. about a 16% reduction in cross-entropy. That sounds tiny — and it is — but it compounds over many orders of magnitude, and small drops in cross-entropy translate into qualitatively different model behaviour at the right scale.

Why this is so important. Most engineering quantities saturate. Add more cores, you eventually hit Amdahl's law. Add more disk, you eventually hit network. Power laws don't saturate; they slow. The Kaplan curves told the field that a model $100\times$ bigger *will* be measurably better, and quantified by how much. That is the licence to spend.

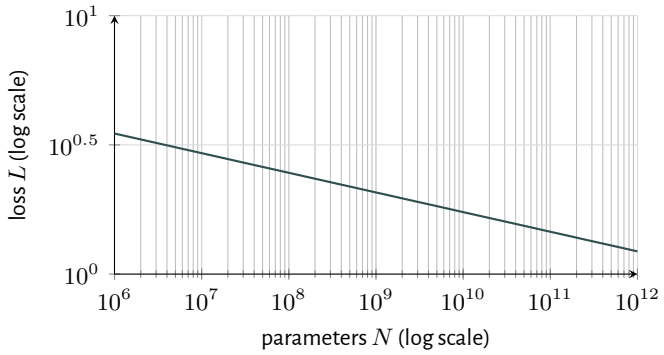

Figure 7.1: The shape of $L(N)$ on log-log axes — straight, with slope $-\alpha_N$. Same picture for $L(D)$ and $L(C)$. Numbers are illustrative; the point is the geometry.

7.3 Architecture details barely matter (within reason)

Another result, almost as important as the lines themselves: depth, width, heads, head dimension, layer normalisation flavour — all of those move the intercept of the line by a small amount, but not the slope. Within a generous envelope, what determines loss is total parameter count, not how those parameters are arranged.

This is why papers comparing two architectures on a single point estimate of model size are often misleading: the difference between, say, GPT-style and BERT-style at 350M parameters tells you almost nothing about which would win at 70B parameters.

7.4 What Kaplan got slightly wrong

The original paper recommended that, given a fixed compute budget, you should make models *larger* faster than you should give them more data. This was the recipe behind GPT-3 (175B parameters, $\sim$300B training tokens — a token-to-parameter ratio of less than 2). Two years later, Chinchilla [15] re-ran the analysis with a wider sweep and found that the optimal allocation shifts the other way: parameters and tokens should grow *together*, with a token-to-parameter ratio of roughly 20.

An illustrative extrapolation. The compact toy form

$$L(N) \;\approx\; L(N_0)\left(\frac{N_0}{N}\right)^{\alpha_N}, \quad \alpha_N = 0.076, \tag{7.5}$$

anchored to GPT-3 ($N_0 \approx 1.75 \times 10^{11}$ parameters) with a reported WebText cross-entropy near 1.7 nats,[2] makes the geometry concrete. Plug in an illustrative "GPT-4-scale" dense budget of $N \approx 1.7 \times 10^{12}$ (1.7T) parameters: the ratio $(N_0/N)^{0.076} \approx 0.10^{0.076} \approx 0.84$, so $L(N) \approx 1.7 \times 0.84 \approx 1.4$ nats. That is not a dramatic drop for a $10\times$ parameter increase, which is exactly why the exponent matters more than the headline parameter count. The story is *illustrative only*: Hoffmann et al. [15] moved the optimum toward many more tokens per parameter, real frontier systems are often mixture-of-experts, and public loss numbers for later models are not comparable one-to-one to WebText nats. The structural point is that a smooth power law can make extrapolation look like deterministic forecasting while hiding the fact that data and routing — not N alone — dominate production behaviour.

[2] Actual frontier runs blend many datasets and recipes; treat $L(N_0)$ as an order-of-one anchor only.

Emergent abilities: claims and corrections. A related debate sits one layer above the loss curves. Wei et al. [42] catalogued sharp jumps on benchmarks once models crossed scale thresholds: tasks look "impossible" until they suddenly aren't. Schaeffer et al. [48] argued that many of those jumps disappear under smoother metrics or continuous skill measurements, so perceived emergence tracks metric discontinuity more than mysterious phase transitions. The honest synthesis is middle-ground: loss curves are smooth while downstream pass/fail metrics sit on thresholds, so nonlinear capability can surface without contradicting smooth optimisation dynamics — especially when evaluation is binarised.

We cover Chinchilla in the next chapter. For now, take Kaplan as the paper that established *that* scaling laws exist and *what shape* they have. The numbers got refined; the framework is permanent.

Key idea. Loss is a power law in parameters, in data, and in compute. The straight lines are why scaling up is a strategy. The exponents tell you how steeply.

Chapter 8

Chinchilla — compute-optimal training

Two years after Kaplan, a team at DeepMind [15] ran a wider sweep over model size N and data D, trained each configuration to convergence, and asked a sharper question. Given a fixed training compute budget C, what is the best joint choice of (N, D)?

8.1 The isoflops surface

Compute, parameters, and tokens are tied by a simple rule of thumb:

$$C \approx 6 \cdot N \cdot D \tag{8.1}$$

FLOPs to train a transformer with N parameters on D tokens.[1]

So an *isoflops curve* (constant C) is a hyperbola $N \cdot D = \text{const}$ in the (N, D) plane. Along this curve, you can buy a bigger model by training it on less data, or vice versa. The Chinchilla question is: where on the curve is the loss lowest?

[1] The factor of 6 is bookkeeping. Forward pass is roughly $2ND$ FLOPs; backward pass is twice that; the constant absorbs "per token, per parameter". Kaplan et al. [14] discuss this carefully.

8.2 The 20:1 rule

The empirical optimum sits along

$$D \approx 20 \cdot N \tag{8.2}$$

for a wide range of compute budgets. Concretely:

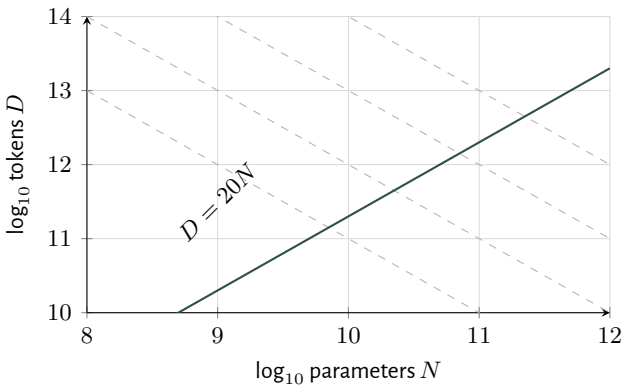

Figure 8.1: Schematic of the Chinchilla finding. Dashed grey lines are isoflops contours ($N \cdot D$ constant). The thick line through the optimum sits at roughly $D = 20N$: for every billion parameters, train on about 20 billion tokens.

Model	Params	Tokens
GPT-3 (Kaplan-pilled)	175B	$\sim$300B
Chinchilla	70B	$\sim$1.4T
Llama 1 (7B)	7B	1T
Llama 2 (7B)	7B	2T
Llama 3 (8B)	8B	15T

The empirical pattern from 2022 onward has been to push the token-to-parameter ratio *above* 20, because at inference time a smaller model is cheaper to serve. Llama 3's 15T tokens per 8B parameters ($\sim$1900:1) reflects that bias: take the loss hit on training efficiency to get a deployment-friendly model.[2]

[2] This is sometimes called "inference-Chinchilla". If you plan to serve the model to billions of requests, the right cost function isn't training compute — it is training compute plus the integral of inference cost over the model's lifetime. That argues strongly for over-training a smaller model.

Worked example: Chinchilla budget for a 100B model. Take $N = 10^{11}$ (100B). The recipe gives $D \approx 20N = 2 \times 10^{12}$ tokens — two trillion tokens seen during training. Equation (8.1) then yields

$$C \approx 6 \cdot (10^{11}) \cdot (2 \times 10^{12}) = 1.2 \times 10^{24} \text{ FLOPs.}$$

At an assumed sustained 500 TFLOP/s per H100 on mixed-precision transformer workloads (5×10^{14} FLOP/s), one device delivers about 1.8×10^{18} FLOPs per hour, so C corresponds to roughly 6.7×10^5 GPU-hours — order 10^6 device-hours before counting failed runs, checkpoint restarts, and evaluation. At a stylised $3 per GPU-hour, that training phase alone is multiple millions of dollars in *rental* cost; published frontier budgets aggregate hardware, power, networking, and iteration overhead well beyond this toy multiply.

8.3 The headline result

The dramatic single experiment in the Chinchilla paper: a 70B model trained on 1.4T tokens *beat* a 280B model (Gopher) trained on 300B tokens, on the same total compute budget. Smaller, more data, better.

Quantity	Value
Tokens $D = 20N$	2×10^{12}
Training FLOPs $C \approx 6ND$	1.2×10^{24}
Single-GPU-hours @ 500 TFLOP/s	$\approx 6.7 \times 10^{5}$
Stylised rental @ \$3/hr	$\mathcal{O}(\$\text{ millions})$

Table 8.1: Illustrative Chinchilla-optimal budget for a 100B-parameter dense LM.

This rewrote the recipe for every frontier lab in 2022–2023. Llama, Mistral, Gemma, Mixtral, Qwen, Phi — every public model since reflects this correction in some form.

8.4 Bridge to long context

Both Kaplan and Chinchilla say that loss is a smooth function of *train-time* resources. They are completely silent on *inference-time* cost — and specifically silent on the question we have been building toward: how much does it cost to *use* a context of length n?

That silence is intentional. The training-loss-versus-resources story holds even with very short contexts; you do not need a 1M-token context window to fit a power law. But the moment you ask the model to actually read a long document at inference time, the costs from Chapter 6 hit, and they are not in the scaling laws at all.

"*Inference-Chinchilla*" — *deployment-bound training*. Textbook Chinchilla fixes (N, D) given a fixed *training* compute envelope. Sardana & Frankle [43] observe that for many shipped products serving — not training — dominates lifetime spend: train once, answer billions of queries. Imagine a deployment with 10^{10} requests, each carrying 10^{3} input tokens — 10^{13} tokens of inference traffic versus 2×10^{12} training tokens above. Even tiny per-token savings from a slightly smaller, slightly-longer-trained model compound across those 10^{10} calls; past a breakpoint, extra training data on the smaller checkpoint buys cheaper inference forever. The arithmetic is qualitative — exact crossover depends on quality constraints, hardware margins, and routing — but it explains why deployment-first teams "over-train" relative to the textbook 20:1 ratio when long-context attention costs (Parts IV–V) dominate serving bills.

That gap is what Chapter 9 is about, and what the rest of this book is fundamentally a response to.

Key idea. Chinchilla says training compute is best spent on smaller models fed more tokens than Kaplan recommended. It does not say anything about how to spend inference compute on long contexts. The latter is the entire problem this book exists to discuss.

Chapter 9

What scaling laws don't tell you

Both Kaplan and Chinchilla are about *training* cost. Their loss curves assume that the model, once trained, can be *used* at the context length it was trained at. That assumption was nearly free in 2020. It is the bottleneck in 2026.

9.1 Three things the laws are silent on

1. *Inference cost as a function of context length.* The Kaplan power law says that loss falls smoothly in parameters N, data D, and training compute C. It does not say a word about how much *compute per query* a trained model needs at sequence length n. That compute grows as $\mathcal{O}(n^2)$ for dense attention (Chapter 6), and that growth has no analog on the scaling-laws plot.

2. *The KV cache.* The cache size grows as $\mathcal{O}(n)$ per layer. For long contexts and large models, it can exceed the model's weight footprint by a wide margin. Scaling laws are written about a model that fits on one GPU and serves one short request at a time; they have nothing to say about a model whose KV cache for a single request requires multi-GPU sharding.

3. *The quality–length curve.* A nominal context window says how many tokens a model accepts. A *functional* context window says how many tokens a model can actually reason over. There is no scaling law that relates the two. The community-standard benchmarks (36, 37) consistently find that nominal windows of 128K tokens have functional windows much smaller, and that the gap depends on architecture in ways that scaling laws do not capture.

9.2 Why the agentic workloads broke the model

Through about 2022 the typical LLM workload was: short user query (~ 100 tokens), short response (~ 500 tokens). Total context ~ 600 tokens. For that regime, dense attention's n^2 is small, the KV cache fits on a single card, scaling laws describe the behaviour you care about, and the world is fine.

The 2023–2026 wave broke that regime. The new workloads:

- A coding agent reading an entire 400k-token monorepo and refactoring across files.
- A legal assistant reading a 200-page contract and resolving cross-references.
- A research summariser reading 50 PDFs and producing a structured literature map.
- A long-running agent session whose "context" is the trace of every prior tool call, plan, and observation.

These do not look like 600-token chats. They look like 100k–1M-token contexts, served at frontier-model quality. And the scaling-laws-era infrastructure — dense transformer, single H100, FlashAttention, GQA — does not deliver them at any reasonable price.

"Lost in the middle" is a usability fact, not a scaling law. Liu et al. [44] showed that long-context models do not treat all positions equally: needle-in-a-haystack-style tasks tend to succeed when the needle sits near the start or end of the prompt and degrade when it sits in the middle, even when the total length is well within the advertised context window. Nominal "128K context" is therefore a *hardware* statement (how many tokens fit in KV) rather than a *reliability* statement about uniform reasoning quality across positions. The practical takeaway for prompt design is to place instructions, tools, and critical facts where models empirically attend more strongly (often the boundaries), and to distrust "just stuff everything in context" workflows for retrieval. Synthetic suites such as RULER [36] and conversational stress tests such as MRCR [37] exist partly because smooth training-loss curves do not reveal these positional failure modes.

An order-of-magnitude agent TCO. Suppose a product serves 10^9 sessions per year, each averaging 2×10^5 prompt tokens (whose KV must be retained or recomputed) plus 5×10^3 generated tokens — that is order 10^{14} served tokens per year, before counting system prompts, retries, or tool transcripts. On a Llama-3 70B-class stack, attention and KV traffic dominate at these lengths: even with FlashAttention-class kernels, prefill FLOPs scale super-linearly in the prompt and the KV resident set

scales linearly in prompt length per active session (Chapter 11). A dense-attention deployment lands in the 10^8–10^9 GPU-hour-per-year class at frontier per-token economics; a $\sim 10\times$ cheaper long-context architecture (Part V: fixed-sparse, state-space, or routed attention) buys headroom by cutting the dominant term in n or in pairs scored. Frontier API pricing in 2026 reflects this directly, mixing per-token list rates with context-tier surcharges: the meter measures memory and quadratic-ish work, not parameter count alone.

Table 9.1: Two-row sanity check: long-context agents tax memory and attention far more than Chinchilla-style training-loss curves suggest.

Deployment sketch	Dominant cost driver (long prompt)	Quantitative annual order
Dense attention + full KV on H100-class GPUs	KV $\propto n$; attention $\propto n^2$ per token	$\mathcal{O}(10^8)$–10^9 GPU-h / yr class
Part V-style stack (same quality target, subquadratic or smaller state)	Linear / near-linear mixers + smaller KV	$\sim 10\times$ lower GPU-h / yr (illustrative)

9.3 Two responses, and the rest of the book

Faced with the gap between what scaling laws describe (training compute) and what 2026's products need (cheap, accurate long-context inference), the field has split into two camps.

Camp 1: don't change the algorithm, change the implementation. FlashAttention (Chapter 10), better KV-cache schemes (Chapter 11), MQA / GQA, paged caches, sliding windows. The asymptote stays n^2; the constants and the memory bills get much better. This is where most production deployments live in 2026.

Camp 2: change the algorithm. State-space models (Chapter 13), long convolutions and linear attention (Chapter 14), hybrids (Chapter 15), and content-dependent sparse mechanisms like SSA (Chapter 17). The asymptote becomes $\mathcal{O}(n)$ or $\mathcal{O}(n \log n)$; the trade-off is that something else about the model has to be different — finite state, fixed sparsity patterns, or proprietary selection routing.

Key idea. Scaling laws describe training-loss returns on training resources. They do not describe inference-time cost as a function of context length, and that gap is the entire reason the post-transformer landscape exists.

9.4 Where the rest of the book goes

Part IV (next) is Camp 1. Part V is Camp 2. Part VI is the 2×2 frame that makes the whole landscape make sense, and the careful read of SubQ's bet inside it.

Part IV

Living with $\mathcal{O}(n^2)$ — engineering mitigations

Chapter 10

FlashAttention — same math, much better hardware story

The first thing to try when faced with a costly algorithm is to see if you have been computing it stupidly. Dao et al. [16] asked exactly this question of dense attention and discovered that, on modern GPUs, the standard implementation was leaving an enormous amount of performance on the table — not because the algorithm was bad, but because the *memory hierarchy* was being abused.

10.1 The GPU memory hierarchy in one paragraph

A modern data-centre GPU (H100, B200) has three relevant memory tiers:

- *HBM* (high-bandwidth memory): the on-package DRAM. 80 GB on H100, ~ 3 TB/s bandwidth.
- *SRAM* (the on-chip shared memory): a few hundred KB per streaming multiprocessor (SM), but *much* faster — bandwidth on the order of 19 TB/s.
- Registers: even faster, smallest of all.

Going to HBM is roughly $6\times$ slower than staying in SRAM. For an operation that is bottlenecked by data movement rather than arithmetic, choosing where to keep your tensors decides your wall clock.

10.2 The standard implementation's mistake

The naive way to implement attention on a GPU is what equation (5.2) literally says:

1. Materialise the matrix $S = QK^\top \in \mathbb{R}^{n \times n}$ in HBM.
2. Apply a row-wise softmax to S, writing A back to HBM.
3. Compute $O = AV$ by reading A and V from HBM.

For $n = 64{,}000$, S is $\sim$16 GB. Writing it out and reading it back twice is the bottleneck — not the arithmetic.

10.3 What FlashAttention does

The FlashAttention insight is a classical one from numerical linear algebra: *tile* the computation, do it in pieces small enough to fit in SRAM, and *never write the intermediate $n \times n$ matrix to HBM*.

The clever bit is doing softmax this way. Naively, softmax needs the full row in order to compute the normalising constant $\sum_j \exp(s_{i,j})$. The standard trick is to compute it in two passes — once to find the max, once to compute the exponentials and the sum. Dao et al. [16] use a streaming, single-pass formulation based on running statistics: as you process tile after tile of K, V, update running max and running sum in registers, and finalise the output for each query tile when its work is done.

10.4 What it costs and what it doesn't

The good news.

- **Memory.** Activation memory drops from $\mathcal{O}(n^2)$ to $\mathcal{O}(n)$. At $n = 64$K, that is the difference between 16 GB and a few MB.
- **Wall clock.** 2–4$\times$ faster end-to-end for typical sequence lengths in 2022; FlashAttention-2 [17] and FlashAttention-3 [18] push this further by exploiting GPU-specific instructions.
- **Numerics.** The output is bit-equivalent to the naive implementation (modulo standard floating-point reduction order). It is *exact* attention, not an approximation.

The bad news.

$$\text{FLOPs} \;=\; \mathcal{O}(n^2 d). \tag{10.1}$$

Same as before. FlashAttention is a constant-factor win, not an asymptotic one. Doubling n still quadruples the work; the work is just done much more efficiently.

Key idea. FlashAttention buys roughly an order of magnitude of practical context length on the same hardware. It does not change the slope of the cost curve. At sufficiently large n, no implementation of dense attention can be fast enough.

10.5 Why this matters for the rest of the book

Two reasons.

It set the practical baseline. "Faster than FlashAttention-2 on a B200" is the de-facto unit of comparison for any new long-context attention mechanism in 2026. When SubQ reports a 52× prefill speedup at 1M tokens (Chapter 18), the baseline is FlashAttention-2 — which is itself an aggressively-optimised implementation of plain dense attention. Beating that by 52× is a real claim.

It clarified what can't *be fixed by implementation alone.* Once your implementation is IO-optimal, what is left is the algorithm. If the algorithm is n^2, the algorithm has to change. That observation — that we have collectively wrung most of the implementation lemon dry — is why the architectural alternatives in Part V get serious attention now, when they were curiosities in 2020.

Listing 10.1: FlashAttention-1 idea: tiled streaming softmax so no full $n \times n$ score matrix materialises in HBM; FA-2/FA-3 refine warp scheduling and Hopper overlaps (paragraph below).

```
# Outer loop: query tiles Q_i that fit in SRAM. Inner
    ↪ loop: K_j,V_j tiles.
O = zeros_like(V)                       # output [n, d_v],
    ↪  resident in HBM
for q_tile in range(0, n, T_q):
    Q_i = load(Q[q_tile:q_tile+T_q])    # HBM -> SRAM
    O_i = zeros(T_q, d_v); stats init    # registers/SRAM
        ↪ for softmax stats
    for kv_tile in range(0, n, T_k):
        K_j = load(K[kv_tile:kv_tile+T_k])    # HBM ->
            ↪ SRAM
        V_j = load(V[kv_tile:kv_tile+T_k])    # HBM ->
            ↪ SRAM
        S_ij = (Q_i @ K_j.T) * scale          # [T_q,T_k],
            ↪  stays in SRAM (never fully HBM)
        # Merge S_ij into running softmax + accumulate
            ↪ softmax(S_ij) @ V_j on-chip.
        # ...
        # (exact ``streaming softmax'' algebra keeps
            ↪ partial max/sum exp and rescales O_i)
    store(O[q_tile:q_tile+T_q], normalise(O_i))  # single
        ↪  write-back per query tile
```

Dao et al. [16] spell out the exact streaming-softmax algebra used inside the inner loop so partial S_{ij} tiles fuse with prior tiles without ever forming the full $n \times n$ map in HBM.

How FA-2 and FA-3 differ from FA-1. FlashAttention-2 [17] keeps the same numerical streaming-softmax recipe but changes *work scheduling*: sequence-length parallelism warps are assigned more evenly, recomputation for backward passes is tightened, and fewer non-matmul instructions sit on the critical path, yielding roughly 1.5–$2\times$ higher throughput on typical shapes without touching the n^2 flop count.

FlashAttention-3 [18] targets Hopper (`fp8` tensor cores, asynchronous `cp.async` copies, and warp-specialised softmax) so that memory movement and instruction issue overlap more aggressively; again the asymptotics stay quadratic, but wall-clock drops another $\mathcal{O}(1)$ factor on hardware where FA-1 was already close to roofline.

Honest bottom line: each generation bought roughly a 1.5–$2\times$ win on real benchmarks when kernels were bound by HBM bandwidth or occupancy, not a change in $\Theta(n^2)$ scaling—that change remains the province of Part V architectures.

Chapter 11

The KV cache and its band-aids

FlashAttention fixes the activation-memory bill of the $n \times n$ attention matrix during a single forward pass. It does not fix the per-token KV cache that grows during autoregressive generation. This chapter is the catalogue of partial fixes for that second bill.

11.1 Why caching is necessary

Recall the autoregressive loop. To generate token $t+1$, the model needs to attend over tokens $1, \dots, t$. To compute attention, it needs their keys and values. Recomputing $K_{1:t}$ and $V_{1:t}$ from scratch at every step would make generation $\mathcal{O}(n^2 d)$ per token, i.e. $\mathcal{O}(n^3)$ for the whole sequence. Caching makes it $\mathcal{O}(nd)$ per token, i.e. $\mathcal{O}(n^2)$ overall — back to the same asymptote, but with a much friendlier constant.

So at every layer, every head, the model maintains running tensors:

$$K^{(\ell,h)} \in \mathbb{R}^{n \times d_{\text{head}}}, \quad V^{(\ell,h)} \in \mathbb{R}^{n \times d_{\text{head}}}. \tag{11.1}$$

Per token of generation, the bytes added are

$$2 \cdot L \cdot H \cdot d_{\text{head}} \cdot s \tag{11.2}$$

where s is bytes per scalar. Earlier we worked the numbers: roughly 0.5 MB per token for a 7B-class model, half a terabyte for 1M tokens.

Everything that follows is an attempt to reduce that constant.

11.2 Multi-Query Attention (MQA)

Shazeer [19] proposed the simplest fix. Use H different *queries*, but share a *single* key and value tensor across all heads:

$$Q_h \in \mathbb{R}^{n \times d_{\text{head}}}, \quad K \in \mathbb{R}^{n \times d_{\text{head}}}, \quad V \in \mathbb{R}^{n \times d_{\text{head}}}. \tag{11.3}$$

The KV cache shrinks by a factor of H. For 32 heads, the per-token cache drops from 0.5 MB to $\sim$16 KB.[1]

[1] Quality typically degrades a few percentage points on standard benchmarks; the interpretation is that the 32 heads were doing genuinely different work and collapsing their keys throws some of that diversity away.

11.3 Grouped-Query Attention (GQA)

Ainslie et al. [20] compromise: split the heads into g groups, share keys and values within each group. $g = 1$ is MQA; $g = H$ is standard MHA. Llama 2 70B and Llama 3 use GQA with $g = 8$. The cache shrinks by H/g; the quality hit is much smaller than MQA's.

Scheme	KV cache scaling	Quality cost
MHA ($g = H$)	full	none (baseline)
GQA ($g = 8$)	$H/8$ smaller	negligible in practice
MQA ($g = 1$)	$H\times$ smaller	1–3 pp on most benchmarks

To make those ratios concrete at frontier context lengths, take a Llama-3 70B-class stack ($L = 80$, $d_{\text{head}} = 128$, $H = 64$) and ask how big the KV cache is for a *single* 1M-token sequence. Bytes scale as $2Ln\,n_{\text{kv}}\,d_{\text{head}}$ in fp16:

KV layout	KV total (GiB, fp16)	H100 (80 GiB) req. / GPU	B200 (192 GiB) req. / GPU
MHA (64 KV heads)	$\approx 2{,}441$	0	0
GQA-8 (Llama-3 70B)	≈ 305	0	0
MQA (1 KV head)	≈ 38.1	$\leq 2^*$	$\leq 5^*$

Table 11.1: KV cache only, one concurrent request at $n = 10^6$ tokens. *Concurrent count is $\lfloor$GPU memory/KV$\rfloor$ ignoring weights, activations, and fragmentation — real serving fits fewer.

GQA-8 is exactly why production 70B models are servable at 128K–1M on *pods* of GPUs rather than on a single device: the KV alone for a million-token single-sequence residency already exceeds one H100. MQA trades representational freedom for capacity; the Jamba-class hybrids of Chapter 15 attack the same wall by mixing in linear-time blocks.

11.4 Paged KV (vLLM)

Kwon et al. [21] treat KV tensors the way an OS treats memory: split each sequence's cache into fixed-size *pages* (blocks of tokens) and maintain a page table

mapping logical token positions to physical blocks. Multiple sequences share the underlying memory pool; pages are added on demand and freed when the sequence ends or is preempted.

Non-contiguous physical placement avoids the classic "reserve one giant rectangle of memory for the worst-case prompt length" fragmentation problem: different sequences grow by allocating new pages, and short sequences do not waste holes sized for someone else's 1M-token tail. Parallel sampling from a shared prompt is handled copy-on-write — child beams share read-only prompt pages until a branch diverges, then duplicate only the suffix pages that differ, exactly analogous to `fork` + COW in Unix kernels.

The win is throughput, not memory per se: paged KV lets you serve many concurrent requests on the same GPU without fragmenting memory or overprovisioning. It is the standard inference layer for Llama-class models in 2026 (vLLM, TGI, TensorRT-LLM).

11.5 Sliding window attention (SWA)

Mistral's signature trick [22]. Instead of attending to all previous tokens, each token attends only to the last w (typically $w = 4096$). The KV cache size becomes $\mathcal{O}(w)$ instead of $\mathcal{O}(n)$. Mistral 7B layered SWA on top of the standard transformer and combined it with the trick that successive layers' windows compose, so the *effective* receptive field at the top layer is $L \cdot w$.[2]

The SWA trade-off is precisely the one we will revisit in Part V: cheap, but the routing is fixed by position, so the model literally cannot see information outside the window without help.

[2] For $L = 32$, $w = 4096$, effective receptive field is roughly 130K tokens. Less than a true 130K-token attention, because the receptive field is convolutional in shape — distant tokens' information has to be relayed through intermediate layers — but a real-world win.

11.6 RoPE, YaRN, and position extrapolation

Rotary position embeddings (RoPE, 12) encode position by rotating the query and key vectors in 2D subspaces by an angle proportional to position. The rotation frequencies θ_k are spaced across a geometric ladder, and RoPE has the elegant property that the dot product $q_i \cdot k_j$ only depends on $j - i$, so it composes naturally with relative attention.

If you train at sequence length n_{train} and want to deploy at $n_{\text{infer}} \gg n_{\text{train}}$, naive RoPE breaks because the rotations land in unfamiliar regions of the angle space — the high-frequency components of the ladder, which complete many cycles over the training horizon, run into *unseen* phases at long n, and attention behaves as if it left the training manifold.

NTK-aware rescaling (popularised in community write-ups by kaiokendev,

bloc97, and others) borrows the Neural Tangent Kernel intuition: shrink effective frequencies so rotations advance more slowly at long positions, trading some fine local discrimination for stability. *Position interpolation* — linearly rescaling position indices — is the simplest baseline in the same family. YaRN [13] generalises the idea with learned or hand-tuned stretching of the frequency ladder (sometimes mixing "attention sinks" and temperature-like scaling) so checkpoints trained at L tokens reach $2L, 8L$, or more with much less perplexity blow-up.

These are not algorithmic changes to attention itself — they are clever position-encoding tweaks that let you stretch the trained model's effective range. They do not change the asymptotic cost, and they do not, by themselves, recover the lost reasoning quality at extreme n: the honest recipe for a new length regime remains to train (or at least adapt) there, then evaluate on long benchmarks such as RULER/MRCR rather than on PPL alone.

11.7 The honest summary

Each of MQA, GQA, paged KV, SWA, and RoPE/YaRN is a real win. Together they are the reason production transformers can serve 128K–1M token contexts at all in 2026. None of them changes the $\mathcal{O}(n^2)$ compute or the fundamental fact that the KV cache grows linearly per layer.

Key idea. Every mitigation in this chapter is a band-aid on an $\mathcal{O}(n^2)$ wound. The patient survives. The disease is still there. At sufficiently large n, or sufficiently demanding quality bars, you have to change the algorithm — which is what the rest of the book is about.

Part V

Subquadratic alternatives

Chapter 12

Fixed-pattern sparse attention

PART IV'S PUNCHLINE was that no matter how cleverly you implement dense attention, the number of FLOPs is $\mathcal{O}(n^2 d)$ per layer because every token has to score against every other token. The shortest line of attack is to ask: *do they really?* If most of the n^2 entries of A are very small, why bother computing them at all?

This chapter is about *fixed-pattern sparse attention*: pick, ahead of time, a sparse pattern of which (token, token) pairs get scored, and zero out the rest. The pattern depends only on *position*, not on the content of the tokens.

12.1 The sparse-transformer family

Sparse Transformer. Child et al. [23] use a fixed pattern that mixes *strided* attention (attend to every k-th position) with *local* attention (attend to a fixed window). This brings cost down to $\mathcal{O}(n\sqrt{n})$ for image generation, where they introduced it.

Longformer. Beltagy et al. [24] use *sliding window + global tokens*. Each token attends to a local window (e.g. 512 neighbours), and a small set of designated "global" tokens (CLS, question tokens, etc.) attend to and are attended-to by everyone. Cost: $\mathcal{O}(n \cdot w)$ per layer for window size w, plus a $\mathcal{O}(n)$ global slice — both linear in n.

Effective receptive field of stacked sliding windows. If each layer only mixes tokens within a causal radius of roughly w positions, then — ignoring residual shortcuts — information from token t reaches $t + L(w-1)$ after L layers in the worst case along the causal chain.[1] For Mistral-7B-style stacks ($L = 32, w = 4096$), a back-of-the-envelope reach is $\mathcal{O}(Lw) \approx 1.3 \times 10^5$ tokens — far beyond a single layer's

[1] Residual connections provide shortcuts so this geometric bound is optimistic for *representation* quality; it is still the right scaling intuition for how far local attention "smears" content.

window, yet still not true global attention. The word "effective" matters: every hop passes LayerNorm, nonlinearities, and other heads, so distant mass is attenuated even when a linear path exists on paper. That is why production models pair sliding windows with a few global or attention-heavy layers (Chapter 15).

BigBird. Zaheer et al. [25] combine local + global + random attention. The randomness is the trick: a small amount of random long-range attention plus enough hops makes the attention graph *universal* (in the sense that any function expressible by a dense transformer is also expressible by enough BigBird layers). Cost is $\mathcal{O}(n)$ again with the right constants.

Figure 12.1: Three fixed-pattern sparse attention masks (causal, lower triangular). Filled cells are computed; empty cells are skipped. Adding a few "global" columns (right) recovers most long-range dependencies at linear cost.

12.2 What fixed-pattern sparse gives up

The asymptote falls from $\mathcal{O}(n^2)$ to $\mathcal{O}(n)$. What is sacrificed?

- **Content-blindness.** The same set of attention pairs is used for every input. "Attend to position $i \pm w$" is fast, but if the relevant token is at distance $2w$, the model has to relay information through intermediate layers — possibly many of them.
- **Theoretical expressivity holes.** Some tasks (associative recall over arbitrary distances, exact-position retrieval) provably require either dense or content-routed attention; sliding windows alone cannot express them in a constant number of layers.
- **Brittleness to long-range surprises.** A 100K-token document might contain a single critical sentence at any position. Sliding windows will miss it; global tokens are a partial fix but require knowing in advance which positions matter.

In practice, fixed-pattern sparse + global tokens is the workhorse of production long-context transformers (Mistral SWA, Llama 3 with extended context, every encoder-only long-document model). It is not the bet of this book's last chapter — that bet is on *content-dependent* sparsity (Chapter 17) — but it is the baseline against which content-dependent sparsity has to justify itself.

The win in code is that the score matrix is never materialised at full $n \times n$ size:

Listing 12.1: Causal sliding-window attention without a full $n \times n$ score matrix (illustrative `for` loops).

```
import math
import torch
import torch.nn.functional as F

def sliding_window_attention(q, k, v, causal=True, window
    ↪ =512):
    # q,k,v: [n, d_h] single head; returns [n, d_h]
    n, dh = q.shape
    out = torch.empty_like(q)
    for t in range(n):
        s_lo = max(0, t - window + 1)
        s_hi = t + 1
        scores = (q[t:t+1] @ k[s_lo:s_hi].T) / math.sqrt(
            ↪ dh)  # [1, span]
        if causal:
            pass
        attn = F.softmax(scores, dim=-1)
        out[t] = attn @ v[s_lo:s_hi]
    return out
```

The body is $\mathcal{O}(nw)$ time with an $\mathcal{O}(w)$ temporary score width per step. Production stacks use `flash-attn`'s fused sliding-window kernels (and related block-sparse paths) so the inner loop lives on GPU warps rather than in Python.

Key idea. Fixed-pattern sparse attention buys $\mathcal{O}(n)$ by deciding the sparsity pattern in advance, based only on position. Routing is content-blind. The asymptote is gorgeous; the failure mode is missing the one sentence in a million-token document that actually matters.

Chapter 13

State-space models and Mamba

The other major family of post-transformer architectures takes a completely different angle. Rather than make attention sparse, it deletes attention entirely and replaces it with a *recurrent* update that compresses the past into a fixed-size hidden state. This is, in spirit, a return to the LSTM (Chapter 3). What makes 2023–2026's state-space models more than a nostalgia trip is a mathematical reformulation that lets that recurrent computation be *trained* as a parallel scan rather than as a sequential loop.

13.1 What a state-space model is

A continuous-time linear state-space model is the differential equation

$$h'(t) \;=\; Ah(t) + Bu(t), \qquad y(t) \;=\; Ch(t), \tag{13.1}$$

where $u(t) \in \mathbb{R}$ is the input signal, $h(t) \in \mathbb{R}^N$ is the hidden state, and A, B, C are learnable matrices. This is undergraduate control theory.

To use it on a discrete sequence $u_1, u_2, \ldots, u_n$, you discretise with a step size Δ:

$$h_t \;=\; \bar{A}\, h_{t-1} + \bar{B}\, u_t, \qquad y_t \;=\; Ch_t, \tag{13.2}$$

where $\bar{A}, \bar{B}$ are derived from A, B, Δ via a standard zero-order-hold formula. Equation (13.2) looks identical to a linear RNN.[1]

HiPPO in plain English. Classical RNNs pick recurrent dynamics at random and hope gradients survive. HiPPO [49] instead constructs a continuous-time A matrix so the state stores coefficients of the *best* low-degree polynomial approximation to the recent input under a chosen measure (Legendre, Laguerre, …). That is why S4-style layers [26] propagate smooth signals across tens of thousands of steps when

[1] In fact, if you set $\bar{A}$ and $\bar{B}$ to be arbitrary learnable matrices and add a non-linearity, you get back a vanilla RNN. The state-space framing buys two things the RNN didn't have: (i) a principled connection to continuous-time signal processing that gives you *HiPPO* initialisations that handle long-range dependencies stably, and (ii) the parallel-scan computation form, next.

vanilla linear RNNs forget almost everything: the recurrence is biased to act like a rolling orthogonal projection of history.

13.2 The convolutional view

Unrolling equation (13.2) from $h_0 = 0$:

$$y_t \;=\; \sum_{k=0}^{t} \underbrace{C\bar{A}^k\bar{B}}_{\bar{K}_k}\, u_{t-k}. \tag{13.3}$$

This is a *convolution* of the input with a kernel $\bar{K} = (\bar{K}_0, \bar{K}_1, \dots, \bar{K}_{n-1})$ of length n. For *linear* state-space models — where $\bar{A}, \bar{B}, \bar{C}$ do not depend on the data — that kernel is the same for every position. You can compute the convolution with a fast Fourier transform in $\mathcal{O}(n \log n)$, or precompute the kernel and apply it as a single matrix-vector multiply.

S4 [26] took this idea seriously, picked an HiPPO-flavoured A that gives the model a long-range memory, and showed it could match transformers on long-range benchmarks at much lower cost.

13.3 Why S4 wasn't quite enough — the "selectivity" missing piece

S4 was beautiful but fell short of transformer quality on language. The reason, identified by Gu & Dao [27], is that the dynamics $(\bar{A}, \bar{B})$ in equations (13.2) were *input-independent* — the model could not change *how it remembered* based on *what* it was reading. That is the entire strength of attention.

13.4 Mamba — selective state spaces

Mamba's contribution is to make $\bar{A}, \bar{B}, \bar{C}$ functions of the input u_t. Concretely, at every time step the model computes

$$\bar{B}_t = f_B(u_t), \quad \bar{C}_t = f_C(u_t), \quad \Delta_t = f_\Delta(u_t), \tag{13.4}$$

and uses these in a per-step update of the form

$$h_t \;=\; \bar{A}_t\, h_{t-1} + \bar{B}_t\, u_t. \tag{13.5}$$

This breaks the convolution view — you can no longer fold a single fixed kernel across the sequence. Instead, the authors implement the selective recurrence as a hardware-efficient parallel scan that processes the sequence in $\mathcal{O}(n)$ work but $\mathcal{O}(\log n)$ depth on a GPU, keeping the state in SRAM throughout.

Listing 13.1: Selective scan *shape* (sequential Python); real Mamba fuses this into a parallel associative scan on GPU [27].

```
import torch
import torch.nn.functional as F

def selective_scan_demo(u, delta, B, C, A):
    # u, delta, B, C: [T, N]; A: [N]; returns per-step
        ↪ scalars [T] (readout demo)
    T, N = u.shape
    h = torch.zeros(N, dtype=u.dtype)
    ys = []
    for t in range(T):
        decay = torch.exp(-F.softplus(delta[t]) * torch.
            ↪ exp(A))
        h = decay * h + B[t] * u[t]
        ys.append((C[t] * h).sum())
    return torch.stack(ys, dim=0)
```

Production code vectorises channels, fuses the scan, and never runs a Python loop per timestep; the listing only shows why "selective" means Δ_t (here via `softplus`) modulates how much past state survives.

What Mamba wins. Linear-time training and inference ($\mathcal{O}(n)$). Constant-memory generation: there is no growing KV cache, just the fixed-size h_t. For very long sequences this is the entire ballgame.

What Mamba gives up. The hidden state h_t is finite. Every token's information has to be compressed into a vector of size N (typically a few hundred). The cleanest stress test is associative recall: imagine K random `key=value` pairs sprinkled through a T-token sequence, then a query token repeating one key — the model must emit the matching value. A transformer can route the query to the correct position in one softmax attention hop. A selective SSM has to funnel the entire prefix through a vector $h_t \in \mathbb{R}^N$; once K exceeds the effective degrees of freedom in that state, collisions and averaging erase exact key–value bindings, the regime studied systematically by MQAR / "Zoology" analyses [50]. Mamba improves earlier SSMs on this axis but does not restore unconstrained $\mathcal{O}(n)$ addressable memory; hybrids that keep a few attention layers (Chapter 15) exist partly to cover this blind spot.

Key idea. Mamba is a linear-time, constant-memory sequence model. It is genuinely different from a transformer, not a faster transformer. Its benchmarks favour it on smooth signals and tasks with locally-relevant context, and it loses to transformers on tasks that require associative recall over arbitrary positions.

13.5 Where this fits in the bigger picture

State-space models give us the bottom-left of the 2×2 frame in Chapter 16: linear-scaling, content-blind in the sense that the state is a fixed-size summary regardless of what it has seen. Their honest trade-off is the one we will see again and again: cheap, but *lossy*, in a way that depends on the task.

Chapter 14

Linear attention and long convolutions

Two more lines of attack on the n^2 cost don't go through either fixed-pattern sparsity or state-space recurrence. They take the attention equation and reformulate it.

14.1 Linear attention via kernel feature maps

Standard attention computes

$$o_t \;=\; \sum_{s \le t} \frac{\exp(q_t \cdot k_s / \sqrt{d_{\text{head}}})}{\sum_{s' \le t} \exp(q_t \cdot k_{s'} / \sqrt{d_{\text{head}}})}\, v_s. \tag{14.1}$$

The exp of a dot product is the kernel that makes everything quadratic — each q_t has to be paired with every k_s. What if we replace it with a kernel $\phi(q) \cdot \phi(k)$ for some explicit feature map $\phi : \mathbb{R}^{d_{\text{head}}} \to \mathbb{R}^m$?

Then, dropping the normaliser briefly,

$$o_t \;=\; \sum_{s \le t} \phi(q_t) \cdot \phi(k_s)\, v_s \;=\; \phi(q_t)^\top \underbrace{\sum_{s \le t} \phi(k_s)\, v_s^\top}_{S_t}. \tag{14.2}$$

The sum $S_t \in \mathbb{R}^{m \times d_{\text{head}}}$ can be maintained incrementally: $S_t = S_{t-1} + \phi(k_t)\, v_t^\top$. One outer product per token, one matvec per token. Total cost: $\mathcal{O}(n \cdot m \cdot d_{\text{head}})$. Linear in n. Katharopoulos et al. [29] dubbed this *linear attention* and showed the equivalence to a fast linear-RNN.

What goes wrong. The choice of ϕ is the whole game. The exact exp kernel cannot be written as a finite $\phi(q) \cdot \phi(k)$ in any low-dimensional space, so every linear-attention proposal is some kind of compromise.

Performer / FAVOR+ in one pass. The Performer [30] rewrites $\text{softmax}(QK^\top/\sqrt{d})V$ as an expectation over random features ω so that $\exp(q^\top k) \approx \mathbb{E}_\omega[\phi(q,\omega)^\top \phi(k,\omega)]$ with *positive* features ϕ, then applies the same linear-time recurrence trick as above. FAVOR+ reduces variance versus naive random Fourier features and comes with finite-sample guarantees, which is rare among "fast attention" heuristics. In language modelling at scale, however, the estimator still under-matches the exact softmax kernel on associative recall and long-range copying — which is why Performers and related kernel linearisations remain niche compared with hybrids that keep a few dense attention layers.

In code, the recurrence above maintains a fixed-rank state S_t:

Listing 14.1: Linear attention via kernel feature map ϕ [29]; state S_t has fixed rank.

```
import torch
import torch.nn.functional as F

def phi(x):
    return F.elu(x) + 1.0

def linear_attention_recurrence(q, k, v):
    # q,k,v: [T, d]; returns y [T, d] (single-head toy)
    S = torch.zeros(d, d)
    z = torch.zeros(d)
    ys = []
    for t in range(q.shape[0]):
        qt, kt, vt = phi(q[t]), phi(k[t]), v[t]
            ↪ # [d]
        S = S + torch.ger(kt, vt)                          #
            ↪  [d,d] += k v^T
        z = z + kt
        num = S.T @ qt                                     #
            ↪  [d]
        den = (qt * z).sum() + 1e-6
        ys.append(num / den)
    return torch.stack(ys, dim=0)
```

The win is entirely that $S_t \in \mathbb{R}^{d\times d}$ stays constant size while softmax attention would carry an ever-growing $t \times t$ map; the loss is that $\phi(q)^\top \phi(k)$ is only a positive-feature approximation to the full dot-product routing of softmax attention. In practice, linear attention underperforms exact attention by a meaningful amount, especially on associative-recall tasks.

Modern revival: gated linear attention. RetNet [31], GLA, and the Mamba-2 family all use linear-attention machinery with gating to recover some of the lost expressivity. They have become a standard ingredient in hybrids (Chapter 15).

14.2 Long convolutions and Hyena

The other reformulation is to drop attention's all-pairs structure and replace it with a *very long, learned convolutional kernel*. Convolutions are linear in n when computed with the FFT ($\mathcal{O}(n \log n)$ per layer for kernel length n).

Hyena [28] composes implicit (parametrised by an MLP), data-dependent (gated by the input), long convolutions to build a sequence mixer that scales subquadratically. The convolutional kernels are *long enough* to span the entire context, addressing the receptive-field limitation of standard CNN-based language models from the 2010s.

Trade-offs.

- **Asymptote.** $\mathcal{O}(n \log n)$ — better than transformers, worse than pure linear-attention, but the FFT constants are friendly.
- **Routing is again content-light.** The kernel is a function of position (and, in Hyena, a gate function of the input), but it does not *select* which positions to attend to in the way a softmax does.
- **Quality.** Hyena and friends are competitive on long-range benchmarks at small scale; the picture at frontier scale is still being written.

Key idea. Linear attention and long convolutions both win the n-dependence by giving up the precise dot-product routing of softmax attention. They tend to live in hybrids rather than as pure replacements.

Chapter 15

Hybrids — having it both ways

Pure architectures (all-attention transformer; all-Mamba SSM; all-Hyena conv) make for clean papers and clean ablations. In production, the empirical winners since 2024 have been *hybrids*: stacks that interleave attention layers with cheaper mixers, getting most of attention's quality at a fraction of attention's cost.

15.1 Why hybrids work

The argument is empirical. A small fraction of layers — often just one in every k — appears to be doing the heavy lifting on tasks that require exact long-range associative recall (looking up a specific fact at a specific position). Replace those with attention. Replace the rest with something linear. The model's quality is dominated by the few attention layers; the cost is dominated by the many linear ones.

15.2 Three representatives

Jamba [32]. Interleaves Mamba blocks with attention blocks and adds a mixture-of-experts feed-forward on top of both, aiming for the best of three worlds: Mamba layers deliver throughput and long convolutions of memory, attention layers restore associative recall and "look anywhere" routing, and MoE widens parameter capacity without paying a full FFN's FLOPs every token. Public checkpoints report $\mathcal{O}(10^1)$B active parameters with larger total (MoE) capacity and competitive RULER long-context scores relative to same-era dense Mixtral-class baselines; exact headline numbers drift with releases, so treat tables in the paper as authoritative. Ablations in Lieber et al. [32] emphasise that *where* attention is placed matters: removing the periodic attention layers hurts tasks that need sudden jumps in

the sequence, while removing Mamba layers hurts throughput and very long convolutional structure. Arguably the first hybrid taken seriously at frontier scale.

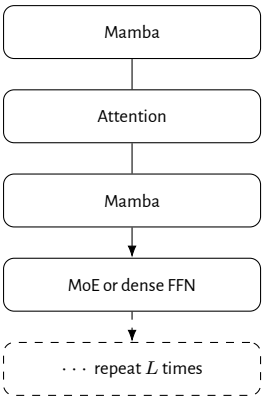

Figure 15.1: Jamba-style layer sandwich (schematic): most depth is linear-time Mamba, with periodic full attention for content-dependent routing [32].

SAMBA [33]. A simpler hybrid: Mamba + sliding-window attention + an MLP, repeated. Trained with effectively unlimited context windows; very strong on long-document benchmarks.

Griffin [34]. Mixes gated linear recurrences (a linear-attention variant) with local attention. Aimed at being a drop-in replacement for transformers in the small-to-mid-size regime.

	Jamba [32]	**SAMBA** [33]	**Griffin** [34]
Layer pattern	Mamba / Attn / Mamba / MoE repeat	Mamba + SWA attention	Linear recurrence (RG-LRU) + local attention
Hidden mixer	Mamba SSM + dense Attn	Mamba + sliding-window Attn	Gated linear + short Conv/Attn
Public scale (order)	52B total / ∼12B active (MoE)	7B-class demos	7B–14B-class recipes
Context / eval	Long-context RULER-style suites	Long seq. via SWA window	Recurrent + local windows
Throughput story†	Few attention layers ⇒ higher tok/s	Windowed attention ⇒ linear-ish	Recurrent core ⇒ small KV
Status	Research + open weights (AI21)	Research (Microsoft)	Research (DeepMind)

Table 15.1: Headline comparison of three hybrid families. †The throughput row is qualitative — vendor-reported or paper-reported claims that depend heavily on hardware and batch; see the original papers for reproducible configs.

15.3 The honest read

Hybrids are the dominant pragmatic answer for "cheap-ish long context without giving up frontier quality" in 2026. The picture they paint is that you cannot, today, completely delete attention from a frontier-quality language model — but you can confine it to a small fraction of layers and let cheaper sequence mixers do everything else.

This is the setup that Subquadratic's bet builds on. The next part is about that bet.

Key idea. Hybrids work because attention's value is concentrated in a few layers per stack, not spread evenly. Replace the cheap layers with linear-time mixers; keep attention for the genuinely hard work.

Part VI

The 2×2 frame and Subquadratic Sparse Attention

Chapter 16

The 2 × 2 frame

If you have read this far you have a list of architectures that all claim to do something interesting with long context: dense transformers, FlashAttention, MQA/GQA, sliding windows, Longformer, BigBird, S4, Mamba, Hyena, Performer, RetNet, Jamba, SAMBA, Griffin, Subquadratic. The list keeps growing. Without a frame to compare them they look like a zoo.

This chapter is the frame. Two axes, four quadrants, every architecture gets a pin.

16.1 The two axes

Axis 1 — Routing. How does the model choose which past information to use at each step?

Position-fixed routing. The set of relevant past tokens is determined entirely by token *positions*, not by their content. "Attend to the last w tokens." "Attend to every k-th position." "Convolve with a fixed kernel." "Maintain a fixed-size running state." Cheap and parallelisable. Bad at queries that need a specific fact at an unpredictable position.

Content-dependent routing. The model uses the current query to *select* which past tokens matter, based on a similarity score between the query and the past tokens' representations. Soft selection (*softmax over all positions*) is what dense attention does. Hard selection (*pick top-k*) is what content-dependent sparse attention does.

Axis 2 — Scaling. What is the asymptotic compute cost as a function of context length n?

Quadratic. $\mathcal{O}(n^2)$. Every pair of tokens scored.

Linear (or near-linear). $\mathcal{O}(n)$ or $\mathcal{O}(n \log n)$. The total compute grows in step with the input.

16.2 The four quadrants

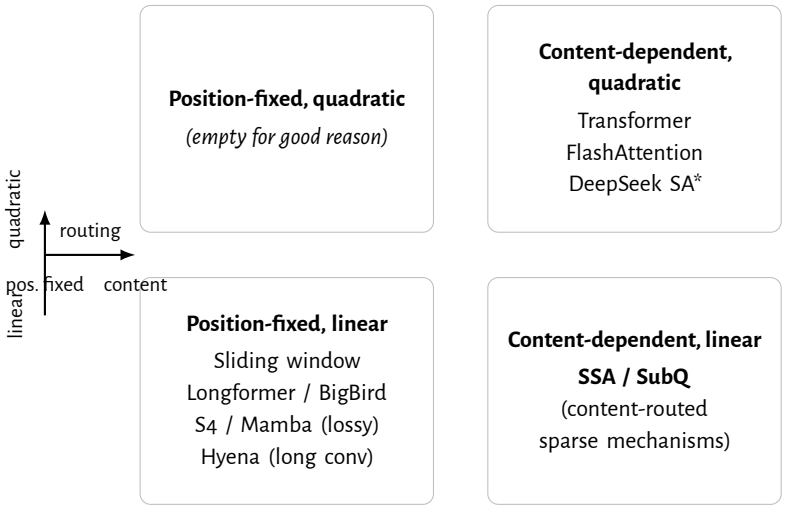

Figure 16.1: The 2×2 frame. Top-right is dense attention and its implementation-only optimisations. Bottom-left is the position-fixed linear-cost zoo. Bottom-right — content-dependent and linear — is the quadrant SSA aims at. Top-left is empty because spending $\mathcal{O}(n^2)$ on a content-blind mechanism is strictly dominated.

Top-left (position-fixed, quadratic). Empty. Spending n^2 compute without using content to choose which pairs to score is strictly worse than dense attention; nobody builds these.

Top-right (content-dependent, quadratic). The classic transformer. FlashAttention, MQA, GQA, paged caches, RoPE/YaRN — every mitigation in Part IV — lives here. The asymptote is n^2; the constants are spectacular by 2026 standards. DeepSeek's recently published DSA [35] sits on the boundary: each query scores a learned *subset* of keys, restoring some of dense attention's content-dependence while cutting *average* FLOPs. Because the router can still admit many pairs in adversarial patterns — and because GPU kernels often densify irregular sparsity — worst-case behaviour stays much closer to quadratic than the clean $\mathcal{O}(n)$ story of fixed windows or convolutions, which is why DSA lands in the upper half of the map rather than in the bottom-right.

Bottom-left (position-fixed, linear). The two big sub-families of Part V live here: fixed-pattern sparse (Chapter 12) and state-space / convolutional models (Chapters 13–14). They are linear in n but content-blind: routing is decided by position or by a fixed kernel, not by the data. This is the dominant flavour of "efficient" attention and it has known weak spots on associative-recall tasks.

Bottom-right (content-dependent, linear). The hard quadrant. You need to keep the dot-product-routing virtue of dense attention, while limiting the number of (query, key) pairs you actually score to be *linear* in n. This requires a non-trivial selection mechanism: given a query, find the small set of keys that matter, in $\mathcal{O}(\log n)$ or $\mathcal{O}(n)$ work, without doing the full $n \times n$ dot product first.

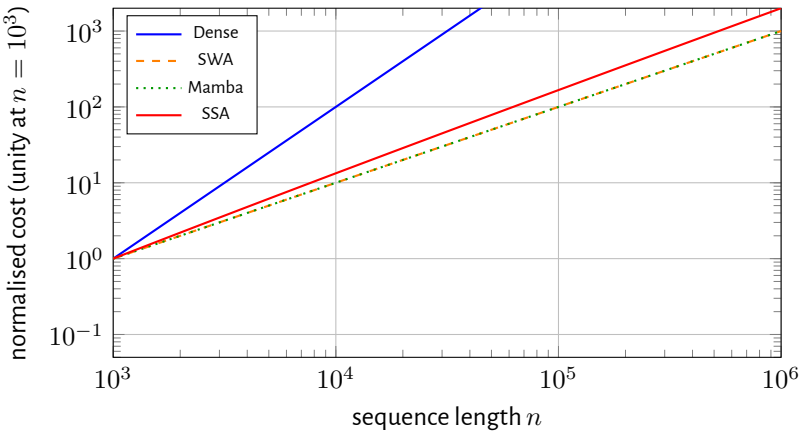

Figure 16.2: Normalised asymptotic shapes: sliding-window and SSM mixers share $\Theta(n)$ after rescaling (orange dashed and green dotted overlap). Dense attention separates upward; an $n \log n$ selector sits between.

Key idea. The interesting frontier in long-context architectures is the bottom-right quadrant: content-dependent routing at linear cost. Every major architecture in Part V occupies a different quadrant; SSA / SubQ explicitly aims for the bottom-right.

Chapter 17

Subquadratic Sparse Attention

Subquadratic Sparse Attention (SSA) is the mechanism behind the SubQ company. This chapter explains, with the limited information that has been made public as of mid-2026, what it is, why it lives in the bottom-right quadrant of Chapter 16, and what is honestly known versus what is still proprietary.[1]

[1] Primary source for this chapter: Subquadratic's technical post [38], supplemented by the company's blog and demos. Where claims are vendor-reported, that is said out loud.

17.1 The mechanism, in one sentence

For each query, score it against every key in the past, but only *actually compute* a small data-dependent subset of those scores — specifically, the ones likely to be largest — and treat the rest as zero.

That subset is chosen by a learned selection function that does not require materialising the full $n \times n$ attention matrix to know which entries matter. The selection function's cost is sublinear or linear in n, not quadratic.

17.2 Why this isn't trivial

The naive way to "only score the top-k keys" is to score *all* keys, sort, and keep the top k. That is $\mathcal{O}(n)$ per query and $\mathcal{O}(n^2)$ per layer — exactly what we are trying to avoid.

The non-trivial bit is finding the top k without doing the all-pairs dot product first. This is the same problem as approximate nearest neighbour (ANN) search, and like ANN it is solved by some combination of:

- **Indexing the keys.** Build a structure (LSH, IVF, HNSW, learned routing tree) that, given a query, returns a small candidate set of keys in $\mathcal{O}(\log n)$ or $\mathcal{O}(\sqrt{n})$

work. Then score only those candidates.

- **Hierarchical selection.** Group keys into clusters at training time; route a query to one or a few clusters; do dense attention within the chosen clusters.
- **Learned routing.** A small network looks at the query and predicts which keys are likely to be relevant, without computing the actual dot products.

The ANN literature is the obvious bag of tricks to lift from. Three families dominate practice: *LSH* hashes nearby points to the same bucket with high probability but trades recall for hash tables that must be rebuilt when keys move every layer; *IVF / k-means partitioning* assigns each key to a Voronoi cell so queries probe only a few cells but needs a good coarse codebook and suffers when the key distribution shifts; *HNSW graphs* offer excellent recall–latency trade-offs on static corpora but are painful to maintain when every training step rewrites all keys. Inside a transformer "the index" is not a static vector store the way FAISS is — it is the entire prefix of the current sequence, changing every token — so differentiable routing, on-the-fly clustering, or hardware-aware tile selection generally beat off-the-shelf ANN libraries copied verbatim from retrieval stacks. Subquadratic [38] describe their selection as content-dependent and hardware-friendly, but the specific algorithm is proprietary.

The toy version of one such selector — bucketed LSH — looks like this:

Listing 17.1: **Illustrative only**—LSH bucketing is one toy selector; SubQ's production SSA router is proprietary and not LSH-specific.

```
import torch
import torch.nn.functional as F

def sparse_attn_lsh_toy(Q, K, V, R, max_bucket=256):
    # Q,K,V have shape (n, d); R has shape (d, b) random
        ↪ projection bits
    n, d = Q.shape
    buckets = {}
    for i in range(n):
        bits = ((R.T @ K[i]) > 0).int()
        idx = int((bits * (2 ** torch.arange(bits.numel()
            ↪ , device=bits.device))).sum())
        key = idx % max_bucket
        buckets.setdefault(key, []).append(i)
    out = torch.empty_like(Q)
    for i in range(n):
        bits = ((R.T @ Q[i]) > 0).int()
        idx = int((bits * (2 ** torch.arange(bits.numel()
            ↪ , device=bits.device))).sum())
        key = idx % max_bucket
        cand = buckets.get(key, list(range(n)))[: min(64,
            ↪  n)]
        scores = (Q[i:i+1] @ K[cand].T) / (d ** 0.5)
```

```
        attn = F.softmax(scores, dim=-1)
        out[i] = attn @ V[cand]
    return out
```

This listing is a teaching toy only. Subquadratic's deployed SSA selection is not public; real systems may use learned routers, IVF/HNSW indices, hybrid reranking, or hardware-aware structures rather than vanilla LSH.

17.3 What this buys you, on paper

- **Cost.** If the selection function returns $\mathcal{O}(\log n)$ or $\mathcal{O}(1)$ keys per query and itself costs $\mathcal{O}(\log n)$ per query, the per-token attention cost is essentially constant in n. Total per-layer cost becomes $\mathcal{O}(n \log n)$ rather than $\mathcal{O}(n^2)$.

- **Routing fidelity.** Unlike sliding-window or state-space models, the selected keys can be *anywhere in the past*. A relevant fact 800K tokens ago can be retrieved as easily as one 8 tokens ago, as long as the selection function thinks to look at it.

- **Drop-in.** The mechanism is meant to slot into a standard transformer's attention block, allowing existing transformer training and inference infrastructure to be reused.

17.4 What it costs in correctness

The honest version is that any sparse-attention mechanism trades off *recall*: the selection function will sometimes miss a key that *should* have been attended to, and the model will not see that information. Two failure modes are worth naming.

Selection-function false negatives. If the selector thinks key s is irrelevant when it actually is, the dependent token t gets no signal from s. The model's behaviour will quietly degrade on tasks where the relevant key looks superficially unrelated to the query.

Training-distribution dependence. The selection function is learned from data. If a long-context use case is sufficiently out-of-distribution, the selector may pick the wrong keys. This is one of the main reasons SubQ-style architectures need extensive long-context-specific training, not just a long-context evaluation tail on top of a standard transformer.

17.5 Where SSA sits relative to the alternatives

In the 2×2 frame from Chapter 16: bottom-right. Linear-cost (or near-linear-cost), content-dependent routing. Distinct from:

- Fixed-pattern sparse (Chapter 12) — same asymptote, position-fixed routing.
- State-space models (Chapter 13) — same asymptote, content-dependent dynamics but *lossy compression* of the past into a finite state.
- Linear attention / Hyena (Chapter 14) — better-than- quadratic asymptote, but expressivity-limited routing kernels.
- Hybrids (Chapter 15) — mostly position-fixed mixers plus a few full-attention layers; complementary to SSA, not competitive.

Key idea. SSA's claim is that it occupies a quadrant nobody else can practically reach: linear-cost *and* content-dependent. If the claim holds at frontier scale, it is a genuine architectural advance, not an implementation trick.

Chapter 18

Reading the benchmarks honestly

Long-context benchmarks have a credibility problem in 2026. Vendors report numbers that look spectacular; independent reproductions either don't exist or contradict the vendor numbers; the underlying evaluation setups (which model, which prompt template, which scoring method, which context window) often differ from one paper to the next.

This chapter is a survival guide for reading SubQ's published numbers without either dismissing them or being sold by them.[1]

[1] Source for all of SubQ's numbers in this chapter: Subquadratic [38]. The "third-party verified" qualification on these numbers refers to a specific evaluation lab; the full methodology was not public as of the last update of this book.

18.1 The three benchmarks that matter

RULER. Hsieh et al. [36] is the de-facto long-context benchmark in 2026. A battery of 13 sub-tasks covering needle-in-a-haystack, multi-key retrieval, multi-hop reasoning, and aggregation, all at configurable context lengths from 4K to 128K. Reported as average accuracy across the 13 tasks at a given length.

RULER variant	What it stresses (one line each; names follow Hsieh et al. [36])
`niah_single_1/2/3`	Three single-needle "haystack" retrievals with different distractor statistics.
`niah_multikey_1/2/3`	Needles that require matching *multiple* keys before the answer is determined.
`niah_multivalue`	One key paired with several candidate values; model must pick the correct binding.
`niah_multiquery`	Several questions about different needles in one long context.
`cwe`	Common-word extraction / counting under distractor text.
`fwe`	Frequent-token statistics where naive bag-of-words heuristics fail without global context.
`vt`	Variable tracking through long procedural text (state must be carried many hops).
`qa_1`	Reading-comprehension QA with long evidence passages (SQuAD-style).
`qa_2`	Multi-hop QA with distractor paragraphs (Hotpot-style reasoning).

Table 18.1: RULER task families (representative naming from the release; exact `json` task ids evolve with leaderboard versions).

The paper bundles *thirteen* scored templates across these families (three single-

key, three multi-key, plus multi-value, multi-query, aggregation, tracing, and QA splits); the table groups them by intent.

MRCR v2. Chen et al. [37] is the harder successor. Multi- document reasoning + retrieval. Tests whether a model can read multiple related documents at long context and answer a question requiring cross-document inference. The 2026 version includes a leaderboard.

Wall-clock prefill speedup. The non-quality benchmark: how fast does the model encode a long prompt? Usually reported relative to a baseline (FlashAttention-2 on a B200, in SubQ's case).

18.2 What SubQ reports

From Subquadratic [38], paraphrased:

Metric	SSA / SubQ result
RULER 128K (avg of 13 tasks)	$\sim$ 95%
Prefill speedup vs FA-2 1M tokens (B200)	$\sim 52\times$
Attention-FLOP reduction vs dense 1M	$\sim$ two orders of magnitude
MRCR v2 (overall)	65.9%

What the RULER number means. 95% on RULER 128K is genuinely strong. Frontier proprietary models (Claude 3.5/4, GPT-4/5, Gemini) report numbers in the 90–96% range at this length; SubQ being competitive here is the headline.

What the prefill speedup means. $52\times$ vs FlashAttention-2 on a B200, at 1M tokens. The baseline matters: FA-2 is itself an aggressive optimisation of dense attention. The speedup is for *prefill* specifically — the initial encoding of the prompt — which is where the n^2 hits hardest. Decode (per-token generation) speedup will be smaller because both architectures hit the KV-cache memory wall in similar ways.

What the MRCR v2 number means — the part the marketing underplays. The honest read of the MRCR v2 leaderboard at the time of writing (paraphrased from 38's own table):

Model	MRCR v2
Claude Opus 4.6	78.3%
GPT-5.5	74.0%
SubQ (SSA)	**65.9%**
GPT-5.4	36.6%
Claude Opus 4.7	32.2%
Gemini 3.1 Pro	26.3%

SubQ is *in the conversation* but *not at the top*. The frontier proprietary models have a clear lead. The relative position is what matters: at a presumably much smaller training-compute footprint, SSA is competitive with — but not yet beating — the best dense transformers at multi-document long-context reasoning. That is a credible architectural result.

18.3 A 13-point checklist for any long-context benchmark report

Before you take a vendor's RULER or MRCR number at face value, walk down this list:

1. Which checkpoint hash / date (not just model marketing name)?
2. Reported context length vs. actual tokens fed (did the harness truncate)?
3. Which benchmark suite (`RULER v0.x`, MRCR v1/v2, …)?
4. Exact-match vs. LLM-as-judge vs. fuzzy match?
5. Fixed prompt template version (Git commit of eval harness)?
6. Greedy decode vs. sampling (temperature, top-p)?
7. Hardware (GPU SKU, driver) and batch size for throughput claims?
8. Vendor self-host vs. independent replicator?
9. Number of samples / CIs (500-sample RULER default vs. smaller smoke tests)?
10. Train vs. eval contamination controls (synthetic only vs. web crawl)?
11. Did the eval include system prompts / tool JSON that inflate token count?
12. For speed claims: prefill vs. decode vs. end-to-end?
13. Are multi-turn or retrieval-augmented variants disabled (apples-to-apples)?

Key idea. Long-context benchmarks are a moving target. SubQ's strongest honest claim is that SSA is competitive with frontier proprietary transformers on RULER and MRCR v2 *while running 50× faster on prefill at 1M tokens*. "Faster than the best, comparable in quality" is a much stronger claim than "best at everything".

18.4 What is missing

A short list of things the public materials do not (yet) settle:

- Training data composition, total training tokens, and parameter count. Without these, comparing to GPT-5 or Claude Opus 4.6 isn't apples-to-apples.
- The full text of the third-party verification.
- Decoding wall-clock numbers (the prefill speedup is for *prompt encoding*; per-token decode performance is a separate question).
- Behaviour at > 1M tokens — RULER 1M, MRCR v2 1M.

The right posture is "cautiously impressed and waiting for reproducibility". That posture is the one this book has tried to model throughout.

Part VII

Where this could go next

Chapter 19

Open questions

No-one has won the long-context architecture argument. This chapter is a short, opinionated list of the questions whose answers will decide it over the next few years.

19.1 Selection algorithms at frontier scale

For content-dependent sparse attention to live in the bottom-right quadrant of Chapter 16, the selection function has to be fast *and* accurate *and* differentiable. We have plenty of algorithms that hit two of those: ANN indexes are fast and accurate but not differentiable; learned routers are differentiable and fast but lose accuracy; hierarchical clustering is differentiable and accurate but slow to update during training. A useful taxonomy of the design space:

- **Data-independent hashing** (LSH, random projections): cheap and free of differentiation, but buckets may miss true nearest neighbours unless the bit width is large — hard when keys change every layer.
- **Data-dependent clustering** (k-means / IVF codebooks): amortises routing lookups, but centroids must track moving keys and degrade under distribution shift.
- **Learned routers** (small MLPs, gating scores): flexible and trainable end-to-end, but risk collapsing diversity if routing saturates (see also DeepSeek-style sparsity [35]).
- **Hierarchical cluster-then-attend**: a coarse stage prunes most keys, a fine stage reranks candidates; good recall but two-pass latency.

- **Hybrid retrieve-and-rerank**: an ANN index proposes candidates and dense dot-products rerank the top-m; closest to production retrieval stacks, still needs careful batching inside autoregressive decoding.

The open question is whether SSA-style mechanisms can keep their selection function quality at the 1B-parameter / 1T-token scale where the real customers live. SubQ's published results suggest yes; independent reproduction is the next move.

19.2 Training infrastructure for the bottom-right quadrant

A standard transformer trains in a well-understood, GPU-friendly way: matrix multiplies dominate, FlashAttention handles the rest, communication patterns are predictable. A content-dependent sparse model breaks every one of those assumptions. The selection function makes computation data-dependent, the sparsity makes communication patterns irregular, and the routing produces load imbalance across devices.

How to train these models at scale, on cheap commodity hardware, with mixed precision and tensor parallelism, is genuinely an open engineering problem. The architectural papers tend to be quiet on this; the implementations that work are mostly proprietary.

19.3 Evaluation that survives Goodhart

Every benchmark in the long-context space has been gamed within months of release.[1] The community's ability to come up with new benchmarks faster than vendors can train to them is the only thing keeping this honest. Expect MRCR v3, v4, etc.

[1] Original needle-in-a-haystack tests were trivial once everyone trained on synthetic NIAH data; the move to RULER and now MRCR v2 is the response. Each new benchmark eventually stops discriminating, and a harder one has to be built.

19.4 Inference economics, not training economics

Frontier model training has consumed the field's budget for five years. The question that decides 2027–2030 is whether anyone can serve a high-quality 1M-token model for under a cent per million input tokens. The arithmetic doesn't work for dense transformers; whether it works for SSA, Mamba-hybrids, or some still-unnamed architecture is genuinely unsettled. Whoever wins the inference economics question wins the agentic-applications business.

A back-of-the-envelope ranking helps ground the discussion. The absolute dollars are not knowable in print, but the relative ratios under identical accounting rules are:

Architecture (same 70B-active class)	$/M input @ 1M ctx (illustrative)	Notes
Dense + FA-2	$\$30$–$\$60$	Quadratic prefill + full KV dominate
GQA + sliding window	$\$8$–$\$20$	Linear-ish layers shrink active pairs; KV still $\mathcal{O}(n)$
Hybrid Mamba–Attn (Jamba-class)	$\$5$–$\$12$	Few attention layers cut peak FLOPs; some full KV blocks remain
SSA / SubQ-style routing	$\$1$–$\$4$	Uses vendor-reported $\sim 50\times$ prefill win vs. FA-2 at 1M as anchor [38]; real cost depends on SLO.

Table 19.1: Order-of-magnitude USD per million input tokens at $n = 10^6$, assuming mixed B200/H100 fleets, utilisation ≈ 0.2–0.4, and blended rent of a few USD per GPU-hour. Meaningful only for comparing ratios between rows.

Key idea. The next five years of LLM research are about inference, not training. Whoever can serve frontier-quality 1M-token context cheaply wins the agent platform business.

Chapter 20

What comes after attention

Sparse attention of any flavour is still attention. The selection function is new; the dot-product-and-softmax routing inside the selected set is unchanged. Several research programmes in 2026 are asking the more aggressive question: *is the entire attention pattern the right computational primitive at all?*

This is speculative. None of the alternatives below has a frontier-quality deployment yet. The point of including them is to remind the reader that the post-transformer landscape might extend well past where this book has mapped it.

20.1 Diffusion language models

Discrete diffusion treats generation as an iterative *denoising* process [40, 51]: a forward corruption (mask tokens, replace with uniform noise, or jump between states) destroys the string, and a learned reverse kernel restores plausible text. The same denoising-diffusion machinery has dominated image generation since 2022; the discrete-token version is younger but follows the same recipe.

Why this could matter. The autoregressive loop is the dominant cost of LLM inference: generating 1000 tokens takes 1000 sequential forward passes. Diffusion steps can be parallelised or scheduled adaptively, and a fluent draft produced in 20 refinement steps is potentially 50$\times$ faster at the same quality.

Why it hasn't won yet. Quality, mostly. Discrete diffusion language models in 2026 (the "Mercury / Inception"-style demos) are competitive with mid-size autoregressive transformers but not with frontier models, and the supporting ecosystem (tool use, streaming UX, KV-style caching) lags the autoregressive stack.

Likelihood evaluation is also less straightforward than next-token negative log-likelihood. The gap may close with scale and with better schedulers; nobody is sure.

20.2 JEPA and the world-model camp

Yann LeCun's Joint Embedding Predictive Architectures [39] argue that LLMs train on the wrong objective: they predict surface symbols (tokens) instead of latent abstractions. A JEPA trains an encoder f to map views of data into a latent space where a small predictor g can forecast *representations* of masked regions from representations of visible context — without reconstructing pixels or tokens directly. Image-JEPA [52] and V-JEPA [53] showed this objective learns semantically meaningful image and video representations for downstream tasks while ignoring high-frequency noise that contrastive losses struggle to filter out.

The bet. If you can train a world model that predicts *abstract* state rather than concrete tokens, you should get a model that plans and reasons better. The open research question for text is whether latent prediction can replace next-token cross-entropy without losing the easy compositional semantics of autoregressive transformers, especially without the multimodal grounding LeCun has emphasised.

Status. Early. As of mid-2026 the commercial stack remains token-autoregressive, but JEPA-style objectives are a credible research path beyond "attention only".

20.3 Programmatic / neuro-symbolic systems

A different bet: instead of making models bigger, make them *call out* to deterministic computation (search engines, code interpreters, databases) and treat the LLM as a *controller* of those tools rather than the source of all answers. Agents in this style are already common; the open question is how much of the LLM's parametric knowledge can eventually be moved to external tools, and whether that lets the LLM itself be much smaller.

20.4 Energy-based and continuous-state models

A small but persistent line of work pursues sequence models that maintain a continuous, low-dimensional state and update it via energy minimisation rather than autoregression. The connection to control theory and physics-informed models is strong; the connection to production NLP is, so far, weak.

20.5 Honest closing

A book like this is by definition out of date the moment it ships. The $\mathcal{O}(n^2)$ wall is real, the engineering response is mature, and the architectural response is still in flux. SSA is one credible bet on the hardest quadrant of that response. Whether it is *the* bet is going to be settled by independent reproduction, by training-cost economics, and by what happens when the next generation of long-context benchmarks makes everyone start over again.

The interesting part of the field is the part where the answer is not yet obvious. We are squarely in that part right now.

Key idea. Attention has been the centre of language modelling for a decade. There is no law that says it has to stay there. The honest position — for engineers, not marketers — is to bet on whichever architecture serves the customer cheapest at the quality the customer needs, and to keep reading the papers.

Chapter A

Tensor shape cheat sheet

A single-page reference for the tensor shapes you'll see in any transformer or transformer-adjacent code. Use it when reading other people's implementations. *Bytes* are for fp16 (2 bytes/element) with batch $B = 1$; multiply by B for larger batches.

Dense transformer (Llama-3 8B–class numerics for illustration)

Take $n = 8{,}192$ tokens, $d_{\text{model}} = d = 4{,}096$, $H = 32$, $d_{\text{head}} = 128$, $d_{\text{ff}} = 14{,}336$, $V = 128{,}256$. FlashAttention never materialises the full $n \times n$ tensor in HBM; the shape still exists *abstractly* for FLOP accounting.

Tensor	Symbol	Shape	Where	Bytes (fp16)
Token ids	x	$[B, n]$	dataloader	$B \cdot n \cdot 4$ (int32)
Embedded tokens	X	$[B, n, d]$	embed lookup	$B \cdot n \cdot d \cdot 2$
Q (one head)	Q_h	$[B, n, d_{\text{head}}]$	projection	≈ 2.0 MiB
K / V (one head)	K_h, V_h	$[B, n, d_{\text{head}}]$	projection	≈ 2.0 MiB each
Attention logits	S_h	$[B, n, n]$	matmul (often *not* stored)	≈ 128 MiB if materialised
Attention weights	A_h	$[B, n, n]$	softmax output	same order
Head output	O_h	$[B, n, d_{\text{head}}]$	$A_h V_h$	≈ 2.0 MiB
MHA block output	—	$[B, n, d]$	concat+proj	$B \cdot n \cdot d \cdot 2$
FFN up-projection	—	$[B, n, d_{\text{ff}}]$	SwiGLU intermediate	≈ 224 MiB
Logits	Z	$[B, n, V]$	lm_head	≈ 2.0 GiB
KV cache (1 layer, MHA)	—	$2 \times [n, d]$	inference RAM	$2 \cdot n \cdot d \cdot 2 \approx 128$ MiB

Table A.1: Order-of-magnitude byte counts for one representative configuration. The attention n^2 cells dominate *activation* memory when stored; FlashAttention avoids storing S_h and A_h wholesale.

The $Q/K/V$ row lists *per head*; multiply by H if you stack before viewing.

KV-cache layout (Llama-3 70B snapshot)

Llama-3 70B uses $L = 80$ layers, $H = 64$, GQA with $n_{\text{kv}} = 8$ repeated key/value heads, $d_{\text{head}} = 128$. Per-token KV footprint scales with $2\, L\, n_{\text{kv}}\, d_{\text{head}}$ rather than $2\, L\, H\, d_{\text{head}}$.

Variant	Key dims / tok	Value dims / tok	fp16 B/tok
MHA (hyp., 64 heads)	$64 \cdot 128$	$64 \cdot 128$	$\approx 65{,}536$
GQA-8 (actual 70B)	$8 \cdot 128$	$8 \cdot 128$	$\approx 8{,}192$
MQA (1 KV head)	128	128	$\approx 1{,}024$

Table A.2: Per-token KV footprint *per layer* scales linearly with the number of KV heads. GQA sits between MHA and MQA; Llama-3 70B uses GQA-8.

Multiply Table A.2 by L and by sequence length n to estimate total cache when serving a prompt.

Selective SSM / Mamba (single stream)

Let state dimension $N = 2048$, input $u_t \in \mathbb{R}$ (per-channel after projection), batch $B = 1$. Tensors below omit batch axes when $B = 1$.

Object	Shape	Note
Hidden state h_t	$[N]$	carried across time; constant memory
Input-dependent Δ_t, B_t, C_t	$[N]$ each	produced from u_t in selective SSM
SSM output y_t	$[N]$	typically projected to model width d
Inference memory	$\mathcal{O}(N)$	no $\mathcal{O}(n)$ KV tensor — contrast with attention

Table A.3: Illustrative shapes for Mamba-style blocks; numeric N varies by checkpoint (e.g. Mamba-2.8B uses a different width than Mamba-130M).

Chapter B

Glossary

A one-line definition for every term-of-art that appears in the book. Alphabetical. Cross-referenced to the chapter where it is introduced.

A–D

activation memory GPU HBM occupied by activations (not weights) during a forward/backward pass; for attention, the $n \times n$ buffers often dominate (Chapter 6).

ANN (approximate nearest neighbour) data structure for finding near neighbours in embedding space in sub-linear time; used in some long-context proposals that route tokens to a subset of past keys.

attention head one parallel low-rank slice of multi-head attention: own Q_h, K_h, V_h projections and one softmax map (Chapter 5).

attention matrix row-softmax of logits $QK^\top/\sqrt{d_{\text{head}}}$; row i is how token i mixes values from all positions.

autoregressive model that factors $p(x_{1:n}) = \prod_t p(x_t \mid x_{<t})$ and is trained to predict the next token (Chapter 1).

BPE (byte pair encoding) greedy merge tokenizer that builds a vocabulary of frequent substrings (Chapter 2).

Bahdanau attention encoder–decoder soft alignment: decoder step attends over all encoder states (Chapter 3).

Chinchilla the compute-optimal scaling recipe from Hoffmann et al. [15]: fix compute budget, prefer smaller models with more tokens than Kaplan-style "big model, fewer tokens" (Chapter 8).

Chinchilla-optimal training point on the iso-compute frontier where parameters N and tokens D satisfy $D \approx 20N$ at large scale (Chapter 8).

content-dependent routing letting network inputs decide which weights or memories to use (e.g. selective SSMs, MoE gates).

convolutional view of SSMs interpreting linear recurrence as a depth-wise convolution along time when parameters are input-independent (Chapter 13).

cross-entropy loss $-\log p_\theta(x_t \mid x_{<t})$ summed over tokens; minimising it is maximum likelihood (Chapter 1).

dense attention every token attends to every other token in the layer's window (full $n \times n$ pattern).

decoder-only transformer stack with causal masking, trained as an LM (GPT-style); contrast encoder-only (BERT) and encoder–decoder (T5).

diffusion language model generative model over tokens or latents that reverses a noise process rather than left-to-right factorisation (Chapter 20).

dropout stochastic zeroing of activations during training as regularisation (mentioned only where training stability matters).

E–L

emergent abilities sharp capability jumps at scale seen only in large models [42]; disputed how "emergent" vs metric-dependent (Chapter 7).

embedding matrix lookup table $E \in \mathbb{R}^{V \times d}$ mapping token ids to vectors (Chapter 2).

FFN (feed-forward network) per-position MLP after attention, usually two linear layers with a non-linearity (Chapter 4).

FlashAttention IO-aware tiling algorithm for attention that reduces HBM traffic [16–18] (Chapter 10).

GQA (grouped-query attention) multiple query heads share one key/value head each group, cutting KV-cache footprint [20] (Chapter 11).

Hyena long-convolution / implicit global filter stack alternative to attention [28] (Chapter 14).

JEPA (Joint Embedding Predictive Architecture) representation learning by predicting latent embeddings from context [39], not next-token logits (Chapter 20).

Kaplan power law empirical scaling trend $L \propto N^{-\alpha}$ etc. from Kaplan et al. [14]; superseded for compute-optimal trade-offs by Chinchilla (Chapter 7).

kernel feature map ϕ such that $k(x, y) = \langle \phi(x), \phi(y) \rangle$; used by Performer to approximate softmax kernels (Chapter 14).

KV cache stored past key/value tensors during autoregressive inference so earlier tokens are not recomputed (Chapter 11).

latent Dirichlet allocation classical Bayesian topic model (mentioned only as contrast with neural LMs).

LayerNorm normalise activations across the feature dimension before each sub-layer (Chapter 4).

linear attention kernelised or reformulated attention whose recurrence is $\mathcal{O}(n)$ in sequence length [29] (Chapter 14).

lost-in-the-middle phenomenon where models under-use tokens in the middle of long prompts [44] (Chapter 9).

LSTM gated RNN cell with separate cell and hidden states to mitigate vanishing gradients [5] (Chapter 3).

M–R

Mamba selective state-space sequence model with hardware-efficient scan [27] (Chapter 13).

Mistral open-weight LM family from Mistral AI [22]; often cited for sliding-window + sparse patterns.

MoE (mixture of experts) feed-forward layer replaced by routed experts so parameter count $\gg$ compute per token.

MQA (multi-query attention) all query heads share a single key head and single value head [19] (Chapter 11).

n-gram model tabulated conditional distributions over fixed-length token histories (Chapter 3).

nucleus sampling sample from smallest token set whose cumulative mass $\geq p$ (Chapter 1).

paged attention non-contiguous KV blocks managed like OS pages to serve many sequences efficiently [21] (Chapter 11).

perplexity exp of average cross-entropy; lower is better (Chapter 1).

position-fixed routing attention or convolution pattern chosen by schedule, not input content (Chapter 12).

Performer attention approximation via random features [30] (Chapter 14).

pre-LN apply LayerNorm before each sub-layer for stabler optimisation (Chapter 4).

RMSNorm root-mean-square normalisation without mean centering; common in Llama-class models.

RNN recurrent hidden state updated each step; $\mathcal{O}(1)$ memory per step but sequential (Chapter 3).

RoPE (rotary position embedding) multiply Q, K by position-dependent rotations [12] (Chapter 4).

RULER synthetic long-context benchmark suite [36] (Chapter 18).

S–Z

SAMBA state-space + attention hybrid architecture [33] (Chapter 15).

SentencePiece unsupervised tokenizer framework using BPE or unigram; shipped with Llama checkpoints [11] (Chapter 2).

sliding-window attention each token attends only to nearby predecessors [24] (Chapter 12).

softmax normalise logits into a probability vector; row-softmax for attention weights (Chapter 5).

sparse attention fixed or learned patterns with sub-n^2 pairs per layer [23, 24] (Chapter 12).

SSA (subquadratic sparse attention) marketing umbrella for routing tokens to a sparse subset for approximate full attention [38] (Chapter 17).

state-space model (SSM) linear dynamical system with structured transitions, sometimes input-selective (Chapter 13).

subword tokenizer output unit smaller than a word (BPE/WordPiece/SentencePiece).

top-k sampling sample only from the k largest logits after optional temperature (Chapter 1).

top-p sampling nucleus sampling: sample from smallest mass-$\geq p$ set (Chapter 1).

transformer stack of self-attention and FFN blocks with residual paths [9] (Chapter 4).

WordPiece subword tokenizer used by BERT with continuation tokens marking word pieces [41] (Chapter 2).

YaRN RoPE scaling variant for longer contexts [13] (Chapter 11).

Chapter C

Annotated reading list

Thirty papers, organised by topic, with one paragraph each on what to read them *for*. Aimed at someone who wants to go deeper than this book and is choosing what to read next.

Foundations: from RNN to Transformer

Elman [3]. Introduces the simple recurrent network as a learnable dynamical system over distributed representations. Read it for the historical proof that "memory" can live in a hidden state updated every step—the baseline recurrence later replaced by attention.

Hochreiter & Schmidhuber [5]. Defines LSTM with forget, input, and output gates so gradients can flow across time through an additive cell state. Read it when you want the authoritative statement of why gating beats vanilla tanh RNNs on long dependencies.

Bahdanau et al. [8]. Adds differentiable soft alignment from decoder to encoder states for sequence-to-sequence MT. Read it to see attention *before* self-attention: weights over positions are learned from two streams of hidden states.

Vaswani et al. [9]. Stacks self-attention and position-wise FFNs, removes recurrence, and argues for parallelism and path length. Read it as the architectural pivot this entire book orbit—everything else measures distance from this baseline.

Devlin et al. [41]. Bidirectional masked-language-model pre-training over Transformer encoders. Read it for encoder-only contrast with GPT-style causal LMs and

for WordPiece tokenisation context.

Scaling: how big and how much

Kaplan et al. [14]. Empirical power laws linking loss to model size, data, and compute; argues for scaling up N within budget. Read it to understand the pre-Chinchilla mental model and why inference costs were not centralised in that era.

Hoffmann et al. [15]. Derives compute-optimal allocation with roughly $20\times$ tokens per parameter at large scale ("Chinchilla"). Read it together with Kaplan to see the explicit trade-off flip toward smaller models trained longer.

Wei et al. [42]. Catalogues tasks where capability jumps appear only beyond a scale threshold. Read it for the empirical phenomenon; pair with sceptical metric-design critiques elsewhere when interpreting "emergence."

Sardana & Frankle [43]. Revisits scaling laws when inference cost matters—not only training FLOPs. Read it as the bridge from Part II scaling laws to serving economics in Part IV.

Engineering: making attention practical

Dao et al. [16]. Tiled attention that reduces HBM round-trips and improves wall-clock without changing numerics. Read it before FA-2/3 so you understand IO-awareness as the core idea.

Dao [17]. Reorders work across warps and improves occupancy for attention on modern GPUs. Read it for the incremental but real throughput wins over FA-1.

Shah et al. [18]. Pushes low-level fusion further on Hopper-class hardware. Read it to see how hardware generations shift what "optimal" kernel design means.

Shazeer [19]. Shares one key and one value head across all query heads to shrink KV cache. Read it as the cleanest memory/compute knob before grouped variants.

Ainslie et al. [20]. Interpolates between MHA and MQA by grouping query heads per KV head. Read it for how Llama-class models recover some expressivity while saving cache.

Kwon et al. [21]. PagedAttention and continuous batching for shared KV storage in inference. Read it when turning memory formulas into a multi-tenant serving story.

Subquadratic alternatives

Beltagy et al. [24]. Combines local sliding windows with global tokens for linear-ish patterns. Read it for fixed sparse patterns (Chapter 12).

Zaheer et al. [25]. Random + window + global sparse attention with theoretical connectivity claims. Read it as another classic fixed-pattern design point.

Gu et al. [26]. Structured state-space sequence layer with stable parameterisation and long range. Read it for the SSM revival before selection and hardware-aware scans.

Gu & Dao [27]. Selective SSM with input-dependent dynamics and efficient parallel scan. Read it for state-space models as a real transformer competitor on throughput.

Poli et al. [28]. Long convolutions and implicit filters as attention replacements. Read it for the convolution / spectral quadrant of the map in Chapter 16.

Katharopoulos et al. [29]. Linearised attention via kernel feature maps and recurrence. Read it for the "linear attention" quadrant without necessarily going all the way to SSMs.

Choromanski et al. [30]. Random-feature approximation for softmax-kernel attention. Read it when you want rigorous sampling-based approximations rather than heuristic sparsity.

Sun et al. [31]. Retention-style recurrence connecting attention and state-space views. Read it for hybrid perspectives that blur Part V categories.

Hybrids and the new wave

Lieber et al. [32]. Alternates transformer layers with Mamba blocks for throughput–quality trade-offs. Read it as a systems-grounded hybrid datapoint.

Ren et al. [33]. Combines Mamba with sliding-window attention for long context. Read it when you want selective sparse routing plus local attention baselines.

De et al. [34]. Gated linear recurrences with local attention in a practical LM recipe. Read it for "RNN flavour" hybrids that still train like modern stacks.

DeepSeek AI [35]. DeepSeek's sparse attention mechanism with empirical long-context results. Read it when placing "DSA" on the routing/subquadratic map (Chapter 17).

Evaluation

Hsieh et al. [36]. Synthetic multi-task long-context benchmark with controllable length. Read it for methodology discipline—what "passes RULER" does and does not prove.

Liu et al. [44]. Shows U-shaped utilisation of context in needle-style tasks. Read it as the counterweight to raw context-length marketing.

Chen et al. [37]. Multi-round conversational retrieval stress test. Read it for evaluation that targets agentic repetition and recall rather than single-shot retrieval.

What might come next

LeCun [39]. World-model flavour of representation learning via latent prediction. Read it when asking whether next-token LM is the only game in town for intelligence.

Lou et al. [40]. Discrete diffusion over tokens as an alternative generative factorisation. Read it for non-autoregressive decoding angles touched on in Chapter 20.

Yao et al. [45]. Interleaves chain-of-thought reasoning with tool actions in text form. Read it for how "LM + retrieval/tools" reframes scaling limits without changing the base model family—complementary to architectural attention debates.

References

Bibliography

[1] R. Kneser and H. Ney. Improved backing-off for m-gram language modeling. *ICASSP*, 1995.

[2] T. Brants, A. Popat, P. Xu, F. Och, and J. Dean. Large language models in machine translation. *EMNLP-CoNLL*, 2007.

[3] J. Elman. Finding structure in time. *Cognitive Science*, 1990.

[4] T. Mikolov, M. Karafiát, L. Burget, J. Černocký, and S. Khudanpur. Recurrent neural network based language model. *INTERSPEECH*, 2010.

[5] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Computation*, 1997.

[6] K. Cho et al. Learning phrase representations using RNN encoder–decoder for statistical machine translation. *EMNLP*, 2014.

[7] I. Sutskever, O. Vinyals, and Q. Le. Sequence to sequence learning with neural networks. *NeurIPS*, 2014.

[8] D. Bahdanau, K. Cho, and Y. Bengio. Neural machine translation by jointly learning to align and translate. *ICLR*, 2015.

[9] A. Vaswani et al. Attention is all you need. *NeurIPS*, 2017.

[10] R. Sennrich, B. Haddow, and A. Birch. Neural machine translation of rare words with subword units. *ACL*, 2016.

[11] T. Kudo and J. Richardson. SentencePiece: a simple and language-independent subword tokenizer. *EMNLP*, 2018.

[12] J. Su et al. RoFormer: enhanced transformer with rotary position embedding. arXiv:2104.09864, 2021.

[13] B. Peng, J. Quesnelle, H. Fan, and E. Shippole. YaRN: efficient context window extension of large language models. arXiv:2309.00071, 2023.

[14] J. Kaplan et al. Scaling laws for neural language models. arXiv:2001.08361, 2020.

[15] J. Hoffmann et al. Training compute-optimal large language models. *NeurIPS*, 2022.

[16] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. FlashAttention: fast and memory-efficient exact attention with IO-awareness. *NeurIPS*, 2022.

[17] T. Dao. FlashAttention-2: faster attention with better parallelism and work partitioning. arXiv:2307.08691, 2023.

[18] J. Shah et al. FlashAttention-3: fast and accurate attention with asynchrony and low-precision. arXiv:2407.08608, 2024.

[19] N. Shazeer. Fast transformer decoding: one write-head is all you need. arXiv:1911.02150, 2019.

[20] J. Ainslie et al. GQA: training generalized multi-query transformer models from multi-head checkpoints. *EMNLP*, 2023.

[21] W. Kwon et al. Efficient memory management for large language model serving with PagedAttention. *SOSP*, 2023.

[22] A. Q. Jiang et al. Mistral 7B. arXiv:2310.06825, 2023.

[23] R. Child, S. Gray, A. Radford, and I. Sutskever. Generating long sequences with sparse transformers. arXiv:1904.10509, 2019.

[24] I. Beltagy, M. Peters, and A. Cohan. Longformer: the long-document transformer. arXiv:2004.05150, 2020.

[25] M. Zaheer et al. Big Bird: transformers for longer sequences. *NeurIPS*, 2020.

[26] A. Gu, K. Goel, and C. Ré. Efficiently modeling long sequences with structured state spaces. *ICLR*, 2022.

[27] A. Gu and T. Dao. Mamba: linear-time sequence modeling with selective state spaces. arXiv:2312.00752, 2023.

[28] M. Poli et al. Hyena hierarchy: towards larger convolutional language models. *ICML*, 2023.

[29] A. Katharopoulos, A. Vyas, N. Pappas, and F. Fleuret. Transformers are RNNs: fast autoregressive transformers with linear attention. *ICML*, 2020.

[30] K. Choromanski et al. Rethinking attention with Performers. *ICLR*, 2021.

[31] Y. Sun et al. Retentive network: a successor to transformer for large language models. arXiv:2307.08621, 2023.

[32] O. Lieber et al. Jamba: a hybrid Transformer-Mamba language model. arXiv:2403.19887, 2024.

[33] L. Ren et al. SAMBA: simple hybrid state space models for efficient unlimited context language modeling. arXiv:2406.07522, 2024.

[34] S. De et al. Griffin: mixing gated linear recurrences with local attention for efficient language models. arXiv:2402.19427, 2024.

[35] DeepSeek AI. DeepSeek Sparse Attention: a content-dependent attention mechanism. 2024 technical report.

[36] C.-P. Hsieh et al. RULER: what's the real context size of your long-context language models? arXiv:2404.06654, 2024.

[37] G. Chen et al. MRCR: a multi-document reasoning and retrieval benchmark. 2025.

[38] Subquadratic. How SSA makes long context practical. `subq.ai/how-ssa-makes-long-context-practical`, 2026.

[39] Y. LeCun. A path towards autonomous machine intelligence. OpenReview, 2022.

[40] A. Lou, C. Meng, and S. Ermon. Discrete diffusion language modelling by estimating the ratios of the data distribution. *ICML*, 2024.

[41] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. BERT: pre-training of deep bidirectional transformers for language understanding. *NAACL*, 2019.

[42] J. Wei et al. Emergent abilities of large language models. *TMLR*, 2022.

[43] N. Sardana and J. Frankle. Beyond Chinchilla-optimal: accounting for inference in language model scaling laws. *arXiv:2401.14426*, 2024.

[44] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang. Lost in the middle: how language models use long contexts. *Transactions of the Association for Computational Linguistics*, 2024.

[45] S. Yao et al. ReAct: synergizing reasoning and acting in language models. *ICLR*, 2023.

[46] K. Clark, U. Khandelwal, O. Levy, and C. Manning. What does BERT look at? An analysis of BERT's attention. *ACL Workshop BlackboxNLP*, 2019.

[47] C. Olsson et al. In-context learning and induction heads. *Transformer Circuits Thread*, 2022.

[48] R. Schaeffer, B. Miranda, and S. Koyejo. Are emergent abilities of large language models a mirage? *NeurIPS*, 2023.

[49] A. Gu, K. Goel, and C. Ré. HiPPO: recurrent memory with optimal polynomial projections. *NeurIPS*, 2020.

[50] S. Arora, S. Eyuboglu, M. Timalsina, et al. Zoology: measuring and improving recall in efficient language models. *ICLR*, 2024. arXiv:2312.04927.

[51] E. Hoogeboom, D. Nielsen, P. Jaini, P. Forré, and M. Welling. Argmax flows and multinomial diffusion: learning categorical distributions. *NeurIPS*, 2021.

[52] M. Assran et al. Self-supervised learning from images with a joint-embedding predictive architecture. *CVPR*, 2023.

[53] A. Bardes, Y. LeCun, and J. Ponce. V-JEPA: self-supervised learning of visual features by video prediction. *arXiv:2406.08589*, 2024.